

 No description has been provided for this image

Python Programming - 2301CS404

Lab - 2

Roll No.: 412

Name: Narola Yash N.

01) WAP to print "Hello World..!!"

```
In [1]: print("Hello World..!!")
```

Hello World..!!

02) WAP to accept your name and display a welcome message.

Input: Priya

Output: Hello Priya, welcome to Python Lab.

```
In [12]: # print(object(S), sep=" ", end="\n", file="Sys.stdout", flush=False)

name = input("Enter your name:")
print("Hello "+name+", welcome to Python Lab.")
print("Hello ", name, ", Welcome to Python Lab.", sep="")
print(id(name))
print(type(name))

# print("Yash" + 18)  TypeError: can only concatenate str (not "int") to str
# print("Yash" + str(18))  Ok...
# print("Yash", 18)  ok...
```

Hello Yash, welcome to Python Lab.

Hello Yash, Welcome to Python Lab.

1729641430208

<class 'str'>

```
In [50]: import sys
```

```
In [53]: name = ""
size = sys.getsizeof(name)
print(size)

name = "Yash"
```

```
size = sys.getsizeof(name)
print(size)
```

41

45

03) WAP to accept three integers and display the numbers, their sum, and average.

Input: 10 20 30

Output:

Numbers: 10 20 30

Sum: 60

Average: 20.0

```
In [6]: num1, num2, num3 = input("Enter three numbers(With space):").split(); # split a
print(num1, num2, num3)
num1 = int(num1)
num2 = int(num2)
num3 = int(num3)
print("Sum: ", num1+num2+num3)
avg = (num1+num2+num3) / 3
print("Average: ", avg)
```

45 55 89

Sum: 189

Average: 63.0

04) WAP to accept name (string), age (int), and percentage (float).

Input : Riya,18,92.5

Output :

Name: Riya Type: <class 'str'>

Age: 18 Type: <class 'int'>

Percentage: 92.5 Type: <class 'float'>

```
In [17]: name, age, pr = input("Enter your name, age and pr (comma seprated):").split(",")
age = int(age)
pr = float(pr)

print(name, "Type:", type(name))
print(age, "Type:", type(age))
print(pr, "Type:", type(pr))
# print(f"Name: {name}: {type(name)}")
# print(f"Age: {age}: {type(age)}")
# print(f"Percentage: {pr}: {type(pr)}")
```

Yash Type: <class 'str'>

18 Type: <class 'int'>

55.5 Type: <class 'float'>

05) WAP to print following message using custom separator and end.

Ooutput : Python | Programming | Basics###

```
In [18]: print("Python", "Programming", "Basics", sep=" | ", end="###")
```

Python | Programming | Basics###

06) WAP to accept a value and display its value, type, and memory id.

Input : hello

Output :

Value: hello

Type: <class 'str'>

ID: 140712345678912

```
In [13]: str = input("Enter the string:")
print("Value:", str)
print("Type:", type(str))
print("ID:", id(str))
```

Value: hello

Type: <class 'str'>

ID: 2199603386032

07) WAP to assign a value to a variable, print id, reassign a new value, and print id again.

Output :

Original ID of a: 140712345678912

New ID of a: 140712345678960

```
In [19]: a = 10
print("Original ID of a:", id(a))
a = 20
print("New ID of a:", id(a))
```

Original ID of a: 140710266320072

New ID of a: 140710266320392

08) WAP to print multiple lines using a single print().

Output:

Welcome to Python

This is the second lab

Enjoy coding!

```
In [16]: print("""Welcome to Python
This is the second lab
Enjoy coding!""")
```

Welcome to Python
This is the second lab
Enjoy coding!

09) WAP to display following table of items with proper alignment.

Output :

Sr No	Name	Subject	Grade	Percentage
1	Nisha Patel	Math	A	92
2	Aarav Modi	Science	B+	85
3	Jiya Shah	English	A+	96

```
In [2]: print(f"{'Yash':5} end")      # in f-string string variable ni alignment by default
print(f"{3525.486:10} end")      # and number ni default alignment Right hase
print(f"{10: 5}")
print(f"{10 + 50j: 20}")
print()

print(f"{'Sr No':^5} {'Name':^12} {'Subject':^8} {'Grade':^5} {'Percentage':^10}")
print(f"{1:5} {'Nisha Patel':12} {'Math':8} {'A':^5} {92:10}")
print(f"{2:5} {'Aarav Modi':12} {'Science':8} {'B+':^5} {85:10}")
print(f"{3:5} {'Jiya Shah':12} {'English':8} {'A+':^5} {96:10}")
```

Yash end
3525.486 end
10
(10+50j)

Sr No	Name	Subject	Grade	Percentage
1	Nisha Patel	Math	A	92
2	Aarav Modi	Science	B+	85
3	Jiya Shah	English	A+	96

10) WAP to accept a float number and display with 2 decimals, 3 decimals, and width 10.

Input : 37.2567

Output :

2 decimals: 37.26

3 decimals: 37.257

Width 10: 37.26

```
In [1]: num = float(input("Input:"))
print(f"2 decimals: {num:.2f}")
```

```
print(f"3 decimals: {num:.3f}")
print(f"Width 10:{num:<10.2f}")
```

2 decimals: 37.26
3 decimals: 37.257
Width 10:37.26

11) WAP to accept two integers and display sum, difference, and product using f-strings.

Input : 12 8

Output :

Sum = 20

Difference = 4

Product = 96

```
In [42]: a, b = input("Enter two number:").split();
a = int(a)
b = int(b)

print(f"Sum = {a+b}")
print(f"Difference = {a-b}")
print(f"Product = {a*b}")

# print("Sum =", a+b)
# print("Difference =", a-b)
# print("Product =", a*b)
```

Sum = 20
Difference = 4
Product = 4

12) WAP to accept date in dd mm yyyy format and display in multiple formats.

Input : 01 12 2025

Output :

01/12/2025

2025-12-01

```
In [59]: dd, mm, yyyy = input("Enter the date (space seprated):").split();
print(dd, mm, yyyy, sep="/")
print(yyyy, mm, dd, sep="-")
```

10/12/2025
2025-12-10

13) WAP to calculate area and perimeter of a circle.

```
In [1]: radius = float(input("Enter radius of the circle: "))
```

```

area = 3.14 * radius * radius
per = 2 * 3.14 * radius

print("Area of the circle:", area)
print("Perimeter (Circumference) of the circle:", "{:.2f}".format(per))

```

Area of the circle: 314.0

Perimeter (Circumference) of the circle: 62.80

14) WAP to convert degree into Fahrenheit and vice versa.

```

In [69]: c = float(input("Enter the Cesium:"))
f = (c*9/5) + 32
print("Fahrenheit:", f)

f = float(input("Enter the Fahrenheit:"))
c = (f-32) * (5/9)
print("Fahrenheit:", c)

```

Fahrenheit: 32.0

Fahrenheit: 0.0

15) WAP to get the distance from user into kilometer, and convert it into meter, feet, inches and centimeter.

```

In [3]: kilo = float(input("Enter the distance in kilometer:"))
meter = kilo * 1000
feet = kilo * 3280.84
inch = kilo * 39,370.1
cm = kilo * 100000

print("Kilometer =", kilo)
print("meter =", meter)
print("feet =", feet)
print("inches =", inch)
print("centimeter =", cm)

```

Kilometer = 1.0

meter = 1000.0

feet = 3280.84

inches = 39370.1

centimeter = 100000.0

```

In [54]: print(locals())
print()
print(globals())

```

```

\nprint(f"{2:>5} {\Aarav Modi\':<12} {\Science\':<8} {\B+\':^5} {85:>10}")\npr
int(f"{3:>5} {\Jiya Shah\':<12} {\English\':<8} {\A+\':^5} {96:>10}")', '_i3
2': 'print(f"\Yash\':5} end") # in f-string string variable ni alinement by
default laft hase\nprint(f"{3525.486:10} end") # and number ni default alinement
Right hase\nprint(f"{10: 5}")\nprint(f"{10 + 50j: 20}")\n\nprint(f"\Sr No\':^5}
{\Name\':^12} {\Subject\':^8} {\Grade\':^5} {\Percentage\':^10}")\nprint(f"
{1:>5} {\Nisha Patel\':<12} {\Math\':<8} {\A\':^5} {92:>10}")\nprint(f"{2:>5}
{\Aarav Modi\':<12} {\Science\':<8} {\B+\':^5} {85:>10}")\nprint(f"{3:>5} {\J
iya Shah\':<12} {\English\':<8} {\A+\':^5} {96:>10}")', '_i33': 'print(f"\Yas
h\':5} end") # in f-string string variable ni alinement by default laft hase
\nprint(f"{3525.486:10} end") # and number ni default alinement Right hase\nprin
t(f"{10: 5}")\nprint(f"{10 + 50j: 20}")\nprint()\n\nprint(f"\Sr No\':^5} {\Nam
e\':^12} {\Subject\':^8} {\Grade\':^5} {\Percentage\':^10}")\nprint(f"{1:>5}
{\Nisha Patel\':<12} {\Math\':<8} {\A\':^5} {92:>10}")\nprint(f"{2:>5} {\Aara
v Modi\':<12} {\Science\':<8} {\B+\':^5} {85:>10}")\nprint(f"{3:>5} {\Jiya Sha
h\':<12} {\English\':<8} {\A+\':^5} {96:>10}")', '_i34': 'num = float(input("In
put:"))\nprint(f"2 decimals: {num:.2f}")\nprint(f"3 decimals: {num:.3f}")\nprint
(f"Width: {num:<10}")', '_i35': 'a, b = input("Enter two number:").split();na =
int(a)\nb = int(b)\n\nprint(f"Sum = {a+b}")\n# print("Sum =", a+b)\n# print("Diff
erence =", a-b)\n# print("Product =", a*b)', 'a': 12, 'b': 8, '_i36': 'a, b = inp
ut("Enter two number:").split();na = int(a)\nb = int(b)\n\nprint(f"Sum = {a+b}")
\nprint(f"Sum = {a-b: 50}")\n# print("Sum =", a+b)\n# print("Difference =", a-b)
\n# print("Product =", a*b)', '_i37': 'a, b = input("Enter two number:").split
();na = int(a)\nb = int(b)\n\nprint(f"Sum = {a+b}")\nprint(f"Sum = {a-b: 5}")\n#
print("Sum =", a+b)\n# print("Difference =", a-b)\n# print("Product =", a*b)', '_
i38': 'a, b = input("Enter two number:").split();na = int(a)\nb = int(b)\n\nprin
t(f"Sum = {a+b}")\nprint(f"Sum = {a-b:5}")\n# print("Sum =", a+b)\n# print("Diffe
rence =", a-b)\n# print("Product =", a*b)', '_i39': 'a, b = input("Enter two numb
er:").split();na = int(a)\nb = int(b)\n\nprint(f"Sum = {a+b}")\nprint(f"Sum = {a
-b}")\n# print("Sum =", a+b)\n# print("Difference =", a-b)\n# print("Product =",
a*b)', '_i40': 'a, b = input("Enter two number:").split();na = int(a)\nb = int
(b)\n\nprint(f"Sum = {a+b}")\nprint(f"Difference = {a-b}")\nprint(f"Product = {a-
b}")\n\n# print("Sum =", a+b)\n# print("Difference =", a-b)\n# print("Product =",
a*b)', '_i41': 'a, b = input("Enter two number:").split();na = int(a)\nb = int
(b)\n\nprint(f"Sum = {a+b}")\nprint(f"Difference = {a-b}")\nprint(f"Product = {a-
b}")\n\n# print("Sum =", a+b)\n# print("Difference =", a-b)\n# print("Product =",
a*b)', '_i42': 'a, b = input("Enter two number:").split();na = int(a)\nb = int
(b)\n\nprint(f"Sum = {a+b}")\nprint(f"Difference = {a-b}")\nprint(f"Product = {a-
b}")\n\n# print("Sum =", a+b)\n# print("Difference =", a-b)\n# print("Product =",
a*b)', '_i43': 'print(locals())', '_i44': 'print(locals())\nprint(globals())', '_
i45': 'print(locals())\nprint()\nprint(globals())', '_i46': 'import Sys', '_i47':
'import sys', 'sys': <module 'sys' (built-in)>, '_i48': 'import sys', '_i49': 'im
port sys', '_i50': 'import sys', '_i51': 'size = sys.getsizeof(name)\nprint(siz
e)', 'size': 45, '_i52': 'name = ""\nsize = sys.getsizeof(name)\nprint(size)', '_
i53': 'name = ""\nsize = sys.getsizeof(name)\nprint(size)\n\nname = "Yash"\nsize
= sys.getsizeof(name)\nprint(size)', '_i54': 'print(locals())\nprint()\nprint(glo
bals())'}

```

```

In [59]: num = 1_234_567 # store number in readalbe form use underscore
print(num)
print(type(num))

```

```

1234567
<class 'int'>

```

```

In [63]: # Complex numbers
a = 45
b = 10
cNum = complex(a, b)
print(cNum)

```

```
cNum = 21+10j
print(cNum)

print("Real part", cNum.real)
print("Imaginary part", cNum.imag)

print(type(cNum))
```

```
(45+10j)
(21+10j)
Real part 21.0
Imaginary part 10.0
<class 'complex'>
```

```
In [65]: # Boolean
bool = True
print(bool, type(bool))

bool = False
print(bool, type(bool))
```

```
True <class 'bool'>
False <class 'bool'>
```

```
In [68]: import keyword
print(keyword.kwlist)

print(type(keyword.kwlist))
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'clas
s', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from',
'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass',
'raise', 'return', 'try', 'while', 'with', 'yield']
<class 'list'>
```

```
In [69]: %reset
```

```
Nothing done.
```


 No description has been provided for this image

Python Programming - 2301CS404

Lab - 3

Roll No.: 412

Name: Narola Yash N.

01) WAP to check whether the given number is positive or negative.

```
In [4]: num = int(input("Enter the number : "))
if num > 0:
    print(f"The {num} is positive.")
elif num < 0:
    print(f"The {num} is negative.")
else:
    print(f"The {num} is zero.")
```

The 0 is zero.

02) WAP to check whether the given number is odd or even.

```
In [6]: num = int(input("Enter the number : "))
if num % 2 == 0:
    print(f"The {num} is even.")
else:
    print(f"The {num} is odd.")
```

The 25 is odd.

03) WAP to find out largest number from given two numbers using simple if and ternary operator.

```
In [2]: a, b = input("Enter two numbers : ").split()
a = int(a)
b = int(b)

if a > b:
    print(f"{a} is largest then {b}")
else:
    print(f"{b} is largest then {a}")

print(f"{a} is largest then {b}") if a>b else print(f"{b} is largest then {a}")
```

541 is largest then 14
541 is largest then 14

```
In [12]: a, b = input("Enter two numbers : ").split()
a = int(a)
b = int(b)

print(f"{a} is largest if a > b else b is largest")
# print(a) if a > b else print(b)
```

8 is largest
8

04) WAP to find out largest number from given three numbers.

```
In [13]: a, b, c = input("Enter three numbers : ").split()
a = int(a)
b = int(b)
c = int(c)

if a > b and a > c:
    print(f"{a} is largest.")
elif b > c:
    print(f"{b} is largest.")
else:
    print(f"{c} is largest.")
```

8 is largest.

```
In [15]: a, b, c = input("Enter three numbers : ").split()
a = int(a)
b = int(b)
c = int(c)

largest = a if a > b and a > c else b if b > c else c
print(f"{largest} is largest")
```

45 is largest

05) WAP to check whether the given year is leap year or not.

[If a year can be divisible by 4 but not divisible by 100 then it is leap year but if it is divisible by 400 then it is leap year]

```
In [21]: year = int(input("Enter the year : "))

if year % 4 == 0:
    if year % 100 == 0:
        if year % 400 == 0:
            print(f"{year} is leap year")
        else:
            print(f"{year} is not leap year")
    else:
        print(f"{year} is leap year")
else:
    print(f"{year} is not leap year")
```

2076 is leap year

06) WAP in python to display the name of the day according to the number given by the user.

```
In [28]: dayNum = input("Enter the number of day (days start from monday) : ")

match dayNum:
    case '1': print("Monday")
    case '2': print("Tuesday")
    case '3': print("Wednesday")
    case '4': print("Thursday")
    case '5': print("Friday")
    case '6': print("Saturday")
    case '7': print("Sunday")
    case _: print("Plz enter valid input")
```

Tuesday

07) WAP to implement simple calculator which performs (add,sub,mul,div) of two no. based on user input.

```
In [34]: a, b = input("Enter two numbers : ").split()
a = float(a)
b = float(b)

op = input("Which operation you want to perform (+, -, *, /) : ")

match op:
    case '+':
        print(f"{a} {op} {b} = {a + b}")
    case '-':
        print(f"{a} {op} {b} = {a - b}")
    case '*':
        print(f"{a} {op} {b} = {a * b}")
    case '/':
        print(f"{a} {op} {b} = {a / b}") if b != 0 else print("divisor is zero")
    case _:
        print("plz enter valid operation")
```

b is zero

08) WAP to read marks of five subjects. Calculate percentage and print class accordingly.

Fail below 35

Pass Class between 35 to 45

Second Class

between 45 to 60

First Class between 60 to 70

Distinction if more than 70

```
In [41]: m1, m2, m3, m4, m5 = input("Enter marks of five subjects : ").split()
m1 = int(m1)
m2 = int(m2)
m3 = int(m3)
m4 = int(m4)
m5 = int(m5)

percentage = (m1 + m2 + m3 + m4 + m5) / 5;
# print(percentage)

if percentage < 35:
    print(f"{percentage} : Fali")
elif percentage >= 35 and percentage < 45:
    print(f"{percentage} : Pass")
elif percentage >= 45 and percentage < 60:
    print(f"{percentage} : Second class")
elif percentage >= 60 and percentage < 70:
    print(f"{percentage} : First class")
elif percentage >= 70 and percentage <= 100:
    print(f"{percentage} : Distinction")
else:
    print("enter valid marks bro")
```

90.0 : Distinction

09) WAP to find the second largest number among three user input numbers.

```
In [43]: a, b, c = input("Enter three numbers : ").split()
a = int(a)
b = int(b)
c = int(c)

if (a > b and a < c) or (a < b and a > c):
    print(f"{a} is second largest")
elif (b > a and b < c) or (b < a and b > c):
    print(f"{b} is second largest")
else:
    print(f"{c} is second largest")
```

45 is second largest

10) WAP to calculate electricity bill based on following criteria. Which takes the unit from the user.

- First 1 to 50 units – Rs. 2.60/unit
- Next 50 to 100 units – Rs. 3.25/unit
- Next 100 to 200 units – Rs. 5.26/unit
- above 200 units – Rs. 8.45/unit

```
In [44]: unit = input("Enter the units: ")
unit = int(unit)
```

```
if unit > 0 and unit <= 50:
```

```
    bill = unit * 2.60
elif unit > 50 and unit <= 100:
    bill = ((unit-50) * 3.25) + (50 * 2.60)
elif unit > 100 and unit <= 200:
    bill = ((unit-100) * 5.26) + (50 * 3.25) + (50 * 2.60)
else:
    bill = ((unit-200) * 8.45) + (100 * 5.26) + (50 * 3.25) + (50 * 2.60)

print(F"unit {unit} : bill {bill}")
```

unit 50 : bill 130.0

 No description has been provided for this image

Python Programming - 2301CS404

Lab - 4

Roll No.: 412

Name: Narola Yash N.

01) WAP to print 1 to 10.

```
In [6]: print(range(1, 11))
        print(type(range(1, 11)))

        for i in range (1, 11):
            print(i)
```

```
range(1, 11)
<class 'range'>
1
2
3
4
5
6
7
8
9
10
```

02) WAP to print 1 to n.

```
In [3]: n = int(input("Enter the number :"))

        for i in range(1, n+1):
            print(i)
```

```
1
2
3
4
5
```

03) WAP to print odd numbers between 1 to n.

```
In [5]: n = int(input("Enter the number :"))

for i in range(1, n+1):
    if i%2!=0:
        print(i)
```

1
3
5
7
9

04) WAP to print numbers between two given numbers which is divisible by 2 but not divisible by 3.

```
In [6]: a = int(input("Enter the first number :"))
b = int(input("Enter the last number :"))

for i in range(a, b+1):
    if i%2==0 and i%3!=0:
        print(i)
```

10
14
16
20

05) WAP to print sum of 1 to n numbers.

```
In [10]: n = int(input("Enter the number : "))

sum = 0
for i in range(1, n+1):
    sum += i

print(sum)
```

15

06) WAP to print sum of series $1 + 4 + 9 + 16 + 25 + 36 + \dots n$.

```
In [11]: n = int(input("Enter the number of steps : "))
sum = 0

for i in range(1, n+1):
    sum += (i * i)

print(sum)
```

55

07) WAP to print sum of series $1 - 2 + 3 - 4 + 5 - 6 + 7 \dots n$.

```
In [12]: n = int(input("Enter the number of steps: "))
sum = 0
```

```

for i in range(1, n+1):
    if i%2 == 0:
        sum -= i
    else:
        sum += i

print(sum)

```

3

08) WAP to print Multiplication Table of the given number.

```

In [17]: n = int(input("Enter the number : "))

for i in range(1, 11):
    print(n, "*", i, "=", n * i)

```

```

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50

```

09) WAP to find Factorial of the given number.

```

In [19]: n = int(input("Enter the number : "))
factorial = 1

for i in range(2, n+1):
    factorial *= i

print(factorial)

```

120

10) WAP to print GCD of given two numbers.

```

In [36]: num1 = int(input("Enter first number : "))
num2 = int(input("Enter second number : "))

min = min(num1, num2)
max = max(num1, num2)

rem = max % min
while rem != 0:
    max = min
    min = rem
    rem = max % min

print(min)

```

4

11) WAP to find Factors of the given number.

In [33]: `num = int(input("Enter the number : "))`

```
for i in range(1, num+1):
    if num%i == 0:
        print(i)
```

1
2
3
4
6
8
12
24

12) WAP to find whether the given number is Prime or not.

In [22]: `num = int(input("Enter the number : "))`
`flag = True`

```
for i in range(2, (num//2)+1):
    if(num % i == 0):
        flag = False
        break
```

```
if flag:
    print(num, "is prime number")
else:
    print(num, "is not prime number")
```

5 is prime number

In [23]: `num = int(input("Enter the number : "))`

```
for i in range(2, (num//2)+1):
    if(num % i == 0):
        print(num, "is not prime number")
        break
else:
    print(num, "is prime number")
```

5 is prime number

13) WAP to print sum of digits of given number.

In [31]: `num = int(input("Enter the number : "))`
`sum = 0`

```
while num != 0:
    sum += num%10
    num //= 10
```

```
print(sum)
```

7

14) WAP to check whether the given number is Palindrome or not.

```
In [24]: num = input("Enter the nubmer : ")

if num == num[::-1]:
    print(num, "is Palindrome")
else:
    print(num, "is not Palindrome")
```

12221 is Palindrome

15) WAP to check whether the given number is an Armstrong Number or not.

```
In [41]: num = int(input("Enter the number : "))
cpy = num

digit = 0
while cpy != 0:
    digit += 1
    cpy //= 10

cpy = num
ans = 0

while cpy != 0:
    prod = 1
    for i in range(digit):
        prod *= (cpy % 10)
    ans += prod
    cpy //= 10

if num == ans:
    print(num, "is armstrong")
else:
    print(num, "is not armstrong")
```

153 is armstrong

16) WAP to print all the perfect numbers between 1 to n.

```
In [66]: num = int(input("Enter the nunmber : "))

for i in range(1, num+1):
    ans = 0
    for j in range(1, i//2 + 1):
        if i%j == 0:
            ans += j

    if i == ans:
        print(i)
```

6
28

```
In [10]: for i in range(1, 5):  
        if i == 10:  
            break  
        else:  
            print("Something")
```

Something

```
In [19]: for i in range(3):  
        for j in range(3):  
            print("(", i, j, ")", end=" ")  
        print()
```

```
( 0 0 ) ( 0 1 ) ( 0 2 )  
( 1 0 ) ( 1 1 ) ( 1 2 )  
( 2 0 ) ( 2 1 ) ( 2 2 )
```

 No description has been provided for this image

Python Programming - 2301CS404

Lab - 5

Roll No.: 412

Name: Narola Yash N.

1) Character Category Counter

Problem: Count uppercase, lowercase, digits, and special characters in a string.

Sample Inputs & Outputs:

- Input: Python3@Lab → Uppercase: 2, Lowercase: 7, Digits: 1, Special: 1
- Input: HELL0123 → Uppercase: 5, Lowercase: 0, Digits: 3, Special: 0
- Input: hi@#9 → Uppercase: 0, Lowercase: 2, Digits: 1, Special: 2

```
In [4]: str = input("Enter the string : ")
Uppercase = Lowercase = Digits = Special = 0

for i in str:
    if i.isupper():
        Uppercase += 1
    elif i.islower():
        Lowercase += 1
    elif i.isdigit():
        Digits += 1
    else:
        Special += 1

print("Uppercase:", Uppercase, "Lowercase:", Lowercase, "Digits:", Digits, "Spec
```

Uppercase: 2 Lowercase: 7 Digits: 1 Special: 1

2) Reverse Each Word

- Input: Python is Easy → nohtyP si ysaE
- Input: Hello World → olleH dlroW
- Input: Learn Python Fast → nraeL nohtyP tsaF

```
In [2]: str = input("Enter the String :")
li = str.split(" ")
```

```

rev = []

for i in li:
    rev.append(i[::-1])

print(" ".join(rev))

```

nohtyP si ysaE

3) Case Pattern Identifier

- PYTHON → Uppercase
- python → Lowercase
- Python Programming → Title Case

```

In [18]: str = input("Enter the string:")

if str.isupper():
    print("UpperCase")
elif str.islower():
    print("LowerCase")
elif str.istitle():
    print("TitleCase")

```

TitleCase

4) First and Last Occurrence Finder

- programming , g → First:3 Last:10
- banana , a → First:1 Last:5
- hello , z → Not found

```

In [20]: str = input("Enter the string: ")
ch = input("Which substring you want to find: ")

firstOccur = str.find(ch)
lastOccur = str.rfind(ch)

print(f"First:{firstOccur} Last:{lastOccur}")

```

First:10 Last:10

5) Word Frequency Counter (ignore the case)

- Python is easy and Python is powerful → python: 2
- Java is popular but java is verbose → java: 2
- C is fast → c:1

```

In [5]: str = input("Enter the string: ")
word = input("Enter the word which you want to count it: ")

print(word, ":", str.lower().count(word.lower()))

```

Python : 2

6) Remove Extra Spaces

- Python is fun → Python is fun
- Hello World → Hello World
- NoExtraSpace → NoExtraSpace

```
In [28]: str = input("Enter the string: ")
str = " ".join(str.split())
print(str)
```

pahgoi isadhg asdgh

7) Prefix and Suffix Removal

- unhappy.txt → happy
- pretest.py → test
- unwanted.doc → wanted

```
In [32]: s = input("Enter the string: ")
pre = input("Enter the prefix: ")
suff = input("Enter the suffix: ")

s = s.removeprefix(pre)
s = s.removesuffix(suff)
print(s)
```

happy

8) Replace Vowels

- Programming → Pr*gr*mm*ng
- Education → *d*c*t**n
- Sky → Sky

```
In [35]: s = input("Enter the string:")

for v in "aieouAIEOU":
    s = s.replace(v, "*");

print(s)
```

*d*c*t**n

9) String Compression

- aaabbccccc → a3b2c4
- xxxyyyz → x3y3z1
- abcd → a1b1c1d1

```
In [12]: s = input("Enter the string: ")

compression = ''
count = 1

for i in range(1, len(s)):
```

```

    if s[i] == s[i-1]:
        count += 1
    else:
        compression += s[i-1] + str(count)
        count = 1

compression += s[-1] + str(count)

print("Compression:", compression)

```

Compression: a1b1c1d1

10) Toggle Case (without using str.swapcase())

- PyThOn → pYtHoN
- HELlo → heLLo
- Python → pYTHON

```

In [38]: s = input("Enter the string: ")
toggle = ''

for i in s:
    if i.isupper():
        toggle += i.lower()
    elif i.islower():
        toggle += i.upper()

print(toggle)

```

pYtHoN

11) Username Validation

valid username rules:

1. only alphabets and digits
2. does not start with a digit
3. length ≥ 6

- User123 → Valid
- 123User → Invalid
- Us@12 → Invalid

```

In [47]: s = input("Enter your username: ")

if not len(s) >= 6:
    print("Invalid")
elif s[0].isdigit():
    print("Invalid")
else:
    for i in s:
        if not i.isalnum():
            print("Invalid")
            break

```

```
else:
    print("valid")
```

valid

12) Palindrome Checker (ignore the case)

- Madam → Palindrome
- Level → Palindrome
- Python → Not Palindrome

```
In [41]: s = input("Enter the string: ")

if s.lower() == s[::-1].lower():
    print("Palindrome")
else:
    print("Not Palindrome")
```

Palindrome

13) Longest Word Finder

- Python programming is interesting → programming
- I love coding → coding
- Data structures and algorithms → structures

```
In [13]: s = input("Enter the string:")
words = s.split()

largestWord = ''
max_len = 0

for i in words:
    if len(i) > max_len:
        max_len = len(i)
        largestWord = i

print(largestWord)
```

Out[13]: ['Python', 'programming', 'is', 'interesting']

14) Case Conversion

- python programming
 - Upper: PYTHON PROGRAMMING
 - Lower: python programming
 - Title: Python Programming
 - Capitalize: Python programming

```
In [16]: s = input("Enter the string:")

print(f"Upper: {s.upper()}")
print(f"Lower: {s.lower()}")
```



```
print(f"Title: {s.title()}")  
print(f"Capitalize: {s.capitalize()}")
```

Upper: PYTHON PROGRAMMING

Lower: python programming

Title: Python Programming

Capitalize: Python programming

15) Custom Split

- apple,banana,grape
- 10|20|30
- A-B-C-D

```
In [19]: s = input("Enter the string:")  
cusSplit = input("Enter the custom split:")  
  
s = s.split()  
s = cusSplit.join(s)  
  
print(s)
```

10|20|30

 No description has been provided for this image

Python Programming - 2301CS404

Lab - 6

Roll No. : 412

Name : Narola Yash N.

01) WAP to find sum of all the elements in a List.

```
In [7]: li = input("Enter the elements:").split()

sum = 0
for i in li:
    sum += int(i)

print("sum of all the elements in a list", sum)
```

sum of all the elements in a list 90

02) WAP to find largest element in a List.

```
In [5]: li = [int(i) for i in input("Enter the elements:").split()]

# 1
# larg = li[0]
# for i in range(1, len(li)):
#     if larg < li[i]:
#         larg = li[i]

# help(list.sort)
# 2
li.sort()
print(li)
larg = li[-1]

print("largest element in a list", larg)
```

[27, 158, 184, 217, 284, 484, 545, 817]

largest element in a list 817

03) WAP to find the length of a List.

```
In [6]: li = [i for i in input("Enter the elements:").split()]

print("Length of a list is", len(li))
```

Length of a list is 2

04) WAP to interchange first and last elements in a list.

```
In [11]: li = [i for i in input("Enter the elements:").split()]
print("before list:",li)
li[0], li[-1] = li[-1], li[0]

print("after list:",li)
```

[12, 12, 854, 12, 84, 45]

05) WAP to split the List into two parts and append the first part to the end.

```
In [12]: li = [i for i in input("Enter the elements:").split()]
print("before list:",li)
part1 = li[:len(li)//2]
part2 = li[len(li)//2:]

part2.extend(part1)

print("after list:", part2)
```

[454, 21, 4, 545]

06) WAP to interchange the elements on two positions entered by a user.

```
In [16]: li = [i for i in input("Enter the elements:").split()]
print("before list:",li)
posi1, posi2 = input("Enter two positions:").split()

li[int(posi1)], li[int(posi2)] = li[int(posi2)], li[int(posi1)]

print("after list:",li)
```

[54, 87, 1, 77, 117, 117, 11]

07) WAP to reverse the list entered by user.

```
In [20]: li = [i for i in input("Enter the elements:").split()]
print("Original list:", li)

li.reverse();

print("Reverse list:", li)
```

[13, 3, 3546, 5451, 45]

08) WAP to print even numbers in a list.

```
In [21]: li = [int(i) for i in input("Enter the elements:").split()]
print("list : ",li)

print("Even number in list: ")
for i in li:
    if i%2 == 0:
        print(i)
```

```
[45, 284, 248, 2148, 124, 1]
Even number in list:
284
248
2148
124
```

09) WAP to count unique items in a list.

```
In [46]: li = [i for i in input("Enter the elements:").split()]
print("list : ",li)

countUni = 0
for i in li:
    if li.count(i) == 1:
        countUni += 1

print("Number of unique items is", countUni)
```

```
list : ['yash', 'narola', 'yash', 'welcome']
Number of unique items is 2
```

10) WAP to copy a list.

```
In [28]: li = [i for i in input("Enter the elements:").split()]
print("list : ",li)

# copyli = li.copy()
# print("copy:", copyli)

# print(id(li))
# print(id(copyli))

# copyli2 = li[:]
# print("copy:", copyli2)

# print(id(li))
# print(id(copyli2))

copyli3 = li
print("copy:", copyli3)
print(id(li))
print(id(copyli3))
```

```
list : ['45', '54', '45']
copy: ['45', '54', '45']
1971426908800
1971426908800
```

11) WAP to print all odd numbers in a given range.

```
In [29]: li = [i for i in range(15) if i % 2 != 0]
print(li)
```

```
[1, 3, 5, 7, 9, 11, 13]
```

12) WAP to count occurrences of an element in a list.

```
In [30]: li = [i for i in input("Enter the elements:").split()]
ele = input("enter element which occurrences you need:")

print("Occurrence of", ele, "is", li.count(ele))
```

```
Occurrence of yash is 2
```

13) WAP to find second largest number in a list.

```
In [40]: li = [int(i) for i in input("Enter the elements:").split()]
li.sort()

print("second largest number is", li[-2])
```

```
second largest number is 85
```

14) WAP to extract elements with frequency greater than K.

```
In [35]: li = [i for i in input("Enter the elements:").split()]
k = int(input("Enter the frequency:"))

newli = []
for i in li:
    if li.count(i) > k and i not in newli:
        newli.append(i)

print("Element with frequency greater than k:", newli)
```

```
Element with frequency greater than k: ['yash', 'nilesh']
```

15) WAP to create a list of squared numbers from 0 to 9 with and without using List Comprehension.

```
In [41]: li = []

for i in range(10):
    li.append(i**2)

print("squared numbers from 0 to 9:", li)
```

```
squared numbers from 0 to 9: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
In [42]: li = [ i**2 for i in range(10) ]
print("squared numbers from 0 to 9:", li)
```

squared numbers from 0 to 9: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

16) WAP to create a new list (fruit whose name starts with 'b') from the list of fruits given by user.

```
In [44]: fruits = [i for i in input("Enter the elements:").split()]

bfruit = [i for i in fruits if i[0] == 'b' or i[0] == 'B']
print("bfruit:", bfruit)
```

bfruit: ['banana', 'blackberry', 'blueberry']

17) WAP to create a list of common elements from given two lists.

```
In [45]: li1 = [i for i in input("Enter the elements:").split()]
li2 = [i for i in input("Enter the elements:").split()]

commonli = []
for i in li1:
    if i in li2 and i not in commonli:
        commonli.append(i)

print("Common elements from given two lists are", commonli)
```

Common elements from given two lists are ['10', '20']

```
In [7]: # Convert input to integers
nums = list(map(int, input().split()))

# map(function, iterable)

# map() vs List comprehension
# Both do same thing, list comprehension is more readable
```

```
In [2]: nums = [1, 2, 3]
result = map(str, nums)
# print(result) -> <map object at 0x0000018495BBB6A0> (returns a map object)
print(list(result))
```

['1', '2', '3']



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 7

Roll No : 412

Name : Narola Yash N.

01) WAP to find sum of tuple elements.

Sample Input:

T = (10, 20, 30, 40)

Sample Output:

Sum of tuple elements = 100

```
In [1]: t = (10, 20, 30, 40)

sum = 0
for i in t:
    sum += i

print(f"Sum of tuple elements = {sum}")
```

Sum of tuple elements = 100

```
In [3]: t = (10, 20, 30, 40)
result = sum(t)
print(f"Sum of tuple elements = {result}")

help(sum)
```

Sum of tuple elements = 100

Help on built-in function sum in module builtins:

```
sum(iterable, /, start=0)
```

Return the sum of a 'start' value (default: 0) plus an iterable of numbers

When the iterable is empty, return the start value.

This function is intended specifically for use with numeric values and may reject non-numeric types.

02) WAP to find Maximum and Minimum K elements in a given tuple.

Sample Input:

```
T = (7, 2, 9, 4, 1, 5)
K = 2
```

Sample Output:

```
Minimum 2 elements: (1, 2)
Maximum 2 elements: (9, 7)
```

```
In [7]: t = (7, 2, 9, 4, 1, 5)
k = int(input("enter the value of k :"))

l = tuple(sorted(t))
print(l)

print(f"Minimum {k} elements: { l[:k] }")
# print(f"Maximum {k} elements: { l[:-k-1:-1] }")
print(f"Maximum {k} elements: { l[-k:][::-1] }")
```

```
(1, 2, 4, 5, 7, 9)
Minimum 2 elements: (1, 2)
Maximum 2 elements: (9, 7)
```

03) WAP to find tuples which have all elements divisible by K from a list of tuples.

Sample Input:

```
tuple_list = [(2, 4, 6), (3, 6, 9), (4, 8, 12), (5, 10, 15)]
K = 2
```

Sample Output:

```
Tuples divisible by 2 : [(2, 4, 6), (4, 8, 12)]
```

```
In [10]: tup = [(2, 4, 6),(3, 6, 9),(4, 8, 12),(5, 10, 15)]
k = int(input("enter the value of k :"))
```



```

# ans = []
# for i in tup:
#     for j in i:
#         if j % k != 0:
#             break
#     else:
#         ans.append(i)

ans = [t for t in tup if all(ele % k == 0 for ele in t)]

print(ans)

help(all)

```

[(2, 4, 6), (4, 8, 12)]

Help on built-in function all in module builtins:

all(iterable, /)

Return True if bool(x) is True for all values x in the iterable.

If the iterable is empty, return True.

04) WAP to create a list of tuples from given list having number and its cube in each tuple.

Sample Input:

```
numbers = [1, 2, 3, 4, 5]
```

Sample Output:

List of tuples with number and its cube: [(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]

```

In [11]: num = [1, 2, 3, 4, 5]

# ans = []
# for i in num:
#     tup = (i, i**3)
#     ans.append(tup)

ans = [(n, n**3) for n in num]

print(ans)

```

[(1, 1), (2, 8), (3, 27), (4, 64), (5, 125)]

05) WAP to find tuples with all positive elements from the given list of tuples.

Sample Input:

```
tuple_list = [(1, 2, 3), (-1, 2, 3), (4, 5, 6), (0, 1, 2), (-3, -4, -5)]
```

Sample Output:

Tuples with all positive elements: [(1, 2, 3), (4, 5, 6)]

```
In [12]: tuple_list = [(1, 2, 3), (-1, 2, 3), (4, 5, 6), (0, 1, 2), (-3, -4, -5)]

# ans = []
# for i in tuple_list:
#     for j in i:
#         if j <= 0:
#             break
#     else:
#         ans.append(i)

ans = [t for t in tuple_list if all(ele > 0 for ele in t)]

print(ans)
```

[(1, 2, 3), (4, 5, 6)]

06) WAP to remove tuples of length K.**Sample Input:**

```
tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
K = 2
```

Sample Output:

List after removing tuples of length 2 : [(3, 4, 5), (6,), (7, 8, 9)]

```
In [13]: tuple_list = [(1, 2), (3, 4, 5), (6,), (7, 8, 9), (10, 11)]
k = int(input("enter the value of k :"))

# ans = []
# for i in tuple_list:
#     if len(i) != 2:
#         ans.append(i)

ans = [t for t in tuple_list if len(t) != k]

print(f"List after removing tuples of length 2: {ans}")
```

List after removing tuples of length 2: [(3, 4, 5), (6,), (7, 8, 9)]

07) WAP to remove all occurrences of a given element from a tuple.**Sample Input:**

```
T = (3, 5, 3, 7, 3, 9)
x = 3
```

Sample Output:

(5, 7, 9)

```
In [15]: T = (3, 5, 3, 7, 3, 9)
k = int(input("enter the value of k :"))

# newT = tuple()
# for i in T:
#     if i != k:
#         newT += (i,)

newT = tuple(ele for ele in T if ele != k)

print(newT)
```

(5, 7, 9)

08) WAP to remove duplicates from tuple.**Sample Input:**

T = (3, 5, 3, 7, 3, 9)

Sample Output:

(3, 5, 7, 9)

```
In [19]: T = (3, 5, 3, 7, 3, 9)

# newT = ()
# for i in T:
#     if i not in newT:
#         newT += (i, )

newT = tuple(dict.fromkeys(T))

print(newT)
```

(3, 5, 7, 9)

09) WAP to multiply adjacent elements of a tuple and print that resultant tuple.**Sample Input:**

t = (2, 3, 4, 5)

Sample Output:

Resultant tuple after multiplying adjacent elements: (6, 12, 20)

```
In [22]: t = (2, 3, 4, 5)

# newT = ()
```

```
# for i in range(len(t) - 1):
#     newT += (t[i] * t[i+1], )

newT = tuple(t[i] * t[i+1] for i in range(len(t) - 1))

print("Resultant tuple after multiplying adjacent elements:", newT)
```

Resultant tuple after multiplying adjacent elements: (6, 12, 20)

10) WAP to test if the given tuple is distinct or not.

Sample Input:

```
tuple1 = (1, 2, 3, 4)
tuple2 = (1, 2, 2, 3)
```

Sample Output:

```
(1, 2, 3, 4) is distinct? -> True
(1, 2, 2, 3) is distinct? -> False
```

```
In [23]: tuple1 = (1, 2, 3, 4)
         tuple2 = (1, 2, 2, 3)

         # for i in tuple1:
         #     if tuple1.count(i) > 1:
         #         print(f"{tuple1} is distinct? -> False")
         #         break
         # else:
         #     print(f"{tuple1} is distinct? -> True")

         # for i in tuple2:
         #     if tuple2.count(i) > 1:
         #         print(f"{tuple2} is distinct? -> False")
         #         break
         # else:
         #     print(f"{tuple2} is distinct? -> True")

         def is_distinct(tup):
             return len(tup) == len(set(tup))

         print(f"{tuple1} is distinct? -> {is_distinct(tuple1)}")
         print(f"{tuple2} is distinct? -> {is_distinct(tuple2)}")
```

```
(1, 2, 3, 4) is distinct? -> True
(1, 2, 2, 3) is distinct? -> False
```

11) WAP to rotate the elements of a tuple to the right by `k` positions.

Sample Input:

```
T = (10, 20, 30, 40, 50)
k = 2
```

Sample Output:

(40, 50, 10, 20, 30)

```
In [39]: T = (10, 20, 30, 40, 50)
k = int(input("enter the value of k :"))

newT = T[-k:] + T[:k+1]
print(newT)
```

(40, 50, 10, 20, 30)

12) WAP to merge two tuples of equal length alternately.

Sample Input:

T1 = (1, 3, 5)
T2 = (2, 4, 6)

Sample Output:

(1, 2, 3, 4, 5, 6)

```
In [40]: T1 = (1, 3, 5)
T2 = (2, 4, 6)

newT = ()
for i in range(len(T1)):
    newT += (T1[i], T2[i])

print(newT)
```

(1, 2, 3, 4, 5, 6)

13) WAP to create a tuple of squares of elements that are even and greater than 10.

Sample Input:

T = (4, 12, 7, 18, 10)

Sample Output:

(144, 324)

```
In [41]: T = (4, 12, 7, 18, 10)

newT = ()
for i in T:
    if i%2==0 and i>10:
        newT += (i**2, )

print(newT)
```

(144, 324)

14) WAP to create (index, value) pairs for prime numbers.

Sample Input:

T = (4, 7, 9, 11, 15)

Sample Output:

((1, 7), (3, 11))

```
In [48]: T = (4, 7, 9, 11, 15)

newT = ()
for i in range(len(T)):
    if T[i] < 2:
        continue
    else:
        for j in range(2, (T[i]//2)+1):
            if T[i] % j == 0:
                break
        else:
            newT += ((i, T[i]),)

print(newT)
```

((1, 7), (3, 11))

15) WAP to split a tuple into even-index and odd-index tuples.

Sample Input:

T = ('a', 'b', 'c', 'd', 'e')

Sample Input:

even = ('a', 'c', 'e')
odd = ('b', 'd')

```
In [50]: T = ('a', 'b', 'c', 'd', 'e')

evenT = ()
oddT = ()

for i in range(len(T)):
    if i % 2 == 0:
        evenT += (T[i], )
    else:
        oddT += (T[i], )

print(f"even = {evenT}")
print(f"oddT = {oddT}")
```

even = ('a', 'c', 'e')
oddT = ('b', 'd')

16) WAP to compute column-wise sum from a tuple of lists.

Sample Input:

```
T = ([1, 2, 3], [4, 5, 6], [7, 8, 9])
```

Sample Output:

```
12 15 18
```

```
In [54]: T = ([1, 2, 3], [4, 5, 6], [7, 8, 9])
```

```
for i in range(len(T)):
    sum = 0
    for j in range(len(T[0])):
        sum += T[j][i]
    print(sum, end=" ")
```

```
12 15 18
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]: tuple -> immutable
```

```
In [3]: t = ()
        print(type(t))
```

```
<class 'tuple'>
```

```
In [ ]: t1 + t2
```

jo ek element tuple ma nakh vo hoy to , karvu pade

```
In [5]: t = (3)
        print(type(t))
```

```
<class 'int'>
```

```
In [6]: t = (3, )
        print(type(t))
```

```
<class 'tuple'>
```

```
In [8]: t = (1, 2, 3, 4)
        t.sort() kam karse nay
        sort() ek j object ni andar chage karni sort kare chhe
        tuple immutable chhe tethi te change nay karva dee
```

```
Cell In[8], line 2
    t.sort() kam karse nay
           ^
SyntaxError: invalid syntax
```

```
In [ ]: list = t.sorted()
```

```
In [ ]:
```


 No description has been provided for this image

Python Programming - 2301CS404

Lab - 8

Roll No : 412

Name : Narola Yash N.

01) WAP to find their union, intersection, and difference of the given two sets.

Sample Input:

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}

```
In [6]: A = set([int(i) for i in input("Enter the element: ").split()])
        B = set([int(i) for i in input("Enter the element: ").split()])

        print("A =", A)
        print("B =", B)

        print("Union:", A.union(B))
        print("Intersection:", A.intersection(B))
        print("Difference (A-B):", A.difference(B))
```

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}

02) WAP to Find elements common to all three given sets.

Sample Input:

A = {1, 2, 3, 4}
B = {2, 3, 5}
C = {2, 3, 6}

Sample Output:

{2, 3}

```
In [10]: A = set([int(i) for i in input("Enter the element: ").split()])  
B = set([int(i) for i in input("Enter the element: ").split()])  
C = set([int(i) for i in input("Enter the element: ").split()])  
  
print(A.intersection(B.intersection(C)))
```

{2, 3}

03) WAP to remove duplicate elements from a list while preserving unique values.**Sample Input:**

[10, 20, 10, 30, 20, 40]

Sample Output:

{10, 20, 30, 40}

```
In [11]: list = [10, 20, 10, 30, 20, 40]  
  
print(set(list))
```

{40, 10, 20, 30}

04) WAP to count how many elements are common between two sets.**Sample Input:**

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

Sample Output:

2

```
In [15]: A = set([int(i) for i in input("Enter the element: ").split()])  
B = set([int(i) for i in input("Enter the element: ").split()])  
  
print(len((A.intersection(B))))
```

05) WAP to Merge two dictionaries. If a key exists in both dictionaries, add their values.

Sample Input:

```
Dict1 = {1: 10, 2: 20, 3: 30}
Dict2 = {2: 40, 3: 50, 4: 60}
```

Sample Output:

```
{1: 10, 2: 60, 3: 80, 4: 60}
```

```
In [4]: Dict1 = {1: 10, 2: 20, 3: 30}
Dict2 = {2: 40, 3: 50, 4: 60}

for k, v in Dict2.items():
    if k in Dict1:
        Dict1[k] += Dict2[k]
    else:
        Dict1[k] = v

print(Dict1)
```

```
{1: 10, 2: 60, 3: 80, 4: 60}
```

06) WAP to Sort a dictionary in ascending order of its values.

Hint: in sorted(), use udf for key parameter

Sample Input:

```
{'Math': 70, 'Physics': 85, 'Chemistry': 60}
```

Sample Output:

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

```
In [12]: dic = {'Math': 70, 'Physics': 85, 'Chemistry': 60}

def getValue(item):
    return item[1]

print(dict(sorted(dic.items(), key=getValue)))
```

```
{'Chemistry': 60, 'Math': 70, 'Physics': 85}
```

07) WAP to find the keys having maximum and minimum values in a dictionary.

Sample Input:

```
{'A': 50, 'B': 90, 'C': 30}
```

Sample Output:

```
Max Value Key: B  
Min Value Key: C
```

```
In [2]: dict1 = {'A': 50, 'B': 90, 'C': 30}  
  
max_val = max(dict1, key=dict1.get)  
min_val = min(dict1, key=dict1.get)  
  
print("Max value Key:", max_val)  
print("Max value Key:", min_val)
```

```
Max value Key: B  
Max value Key: C
```

08) WAP to create a dictionary with numbers from 1 to N as keys and their squares as values.**Sample Input:**

```
N = 5
```

Sample Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
In [14]: dict1 = {}  
for i in range(1, 6):  
    dict1[i] = i ** 2  
  
print(dict1)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

09) WAP to count the frequency of each character in a string using a dictionary.**Sample Input:**

```
programming
```

Sample Output:

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

```
In [16]: string = "programming"  
dict1 = {}
```

```

for i in string:
    if i in dict1:
        dict1[i] += 1
    else:
        dict1[i] = 1

print(dict1)

```

```
{'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

10) WAP to swap keys and values of a dictionary.

Sample Input:

```
{1: 'A', 2: 'B', 3: 'C'}
```

Sample Output:

```
{'A': 1, 'B': 2, 'C': 3}
```

```

In [19]: dict1 = {1: 'A', 2: 'B', 3: 'C'}
dict2 = {}

for k, v in dict1.items():
    dict2[v] = k

print(dict2)

```

```
{'A': 1, 'B': 2, 'C': 3}
```

11) WAP to find the student with the highest total marks, given student names and their marks.

Sample Input:

```
{'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}
```

Sample Output:

```
Topper: Amit
```

```
In [20]: stu_mark = {'Amit': [70, 80, 90], 'Riya': [85, 75, 80], 'Neha': [60, 70, 65]}
highestMarks = 0
highestMarksKey = ""

for k, v in stu_mark.items():
    sum = 0
    for i in v:
        sum += i
    if sum > highestMarks:
        highestMarks = sum
        highestMarksKey = k

print("Topper:", highestMarksKey)
```

Topper: Amit

12) WAP to extract keys whose values are even numbers.

Sample Input:

```
{1: 11, 2: 20, 3: 15, 4: 40}
```

Sample Output:

```
[2, 4]
```

```
In [22]: dict1 = {1: 11, 2: 20, 3: 15, 4: 40}
li = []

for k, v in dict1.items():
    if v % 2 == 0:
        li.append(k)

print(li)
```

[2, 4]

13) WAP to Find common keys between two dictionaries.

Sample Input:

```
Dict1 = {1: 'A', 2: 'B', 3: 'C'}
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}
```

Sample Output:

```
[2, 3]
```

```
In [25]: Dict1 = {1: 'A', 2: 'B', 3: 'C'}
Dict2 = {2: 'X', 3: 'Y', 4: 'Z'}

dict1_key = set(Dict1.keys())
dict2_key = set(Dict2.keys())
```

```
print(list(dict1_key.intersection(dict2_key)))
```

[2, 3]

14) WAP to handle missing keys in dictionaries.

Sample Input - 1:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}
key = 'c'
```

Sample Output - 1:

8

Sample Input - 2:

```
dict1 = {'a': 5, 'c': 8, 'e': 2}
key = 'd'
```

Sample Output - 2:

Key Not Found

```
In [28]: dict1 = {'a': 5, 'c': 8, 'e': 2}
# key = 'c'
key = 'd'

print(dict1.get(key, "Key Not Found"))
```

Key Not Found

15) WAP to create a dictionary using dictionary comprehension from two lists containing student names and marks. Include only those students in the dictionary whose marks are greater than or equal to 75.

Sample Input:

```
students = ['Amit', 'Riya', 'Neha', 'Karan']
marks = [78, 72, 85, 60]
```

Sample Output:

{'Amit': 78, 'Neha': 85}

```
In [32]: students = ['Amit', 'Riya', 'Neha', 'Karan']
marks = [78, 72, 85, 60]

student_marks = {k:v for k, v in zip(students, marks) if v >= 75}

print(student_marks)
```

```
{'Amit': 78, 'Neha': 85}
```


 No description has been provided for this image

Python Programming - 2301CS404

Lab - 9

Roll No : 412

Name : Narola Yash N.

**01) Write a function to calculate BMI given mass and height.
(BMI = mass/h**2)**

```
In [4]: mass = float(input("Enter your weight (kg):"))
height = float(input("Enter your height (m):"))

def calBMI(mass, height):
    return mass / (height ** 2)

print(round(calBMI(mass, height), 2))
```

19.88

02) Write a function that add first n numbers.

```
In [6]: def sumOfFirstN(n):
        return sum(i for i in range(1, n+1))

sumOfFirstN(5)
```

Out[6]: 15

03) Write a function that returns 1 if the given number is Prime or 0 otherwise.

```
In [23]: def isprime(n):
        return 0 if len([i for i in range(2, n//2 + 1) if n%i == 0]) else 1

isprime(17)
```

Out[23]: 1

04) Write a function that returns the list of Prime numbers between given two numbers.

```
In [27]: def isprime(n):  
         return 0 if len([i for i in range(2, n//2 + 1) if n%i == 0]) else 1  
  
         print(list(filter(isprime, range(5, 100))))
```

```
[5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
```

05) Write a function that returns True if the given string is Palindrome or False otherwise.

```
In [32]: def ispalindrome(strr):  
         return True if strr.lower() == strr[::-1].lower() else False  
  
         print(ispalindrome("Yash"))  
         print(ispalindrome("YaaY"))  
         print(ispalindrome("Yaay"))
```

```
False  
True  
True
```

06) Write a function that returns the sum of all the elements of the list.

```
In [33]: def sumOfListEle(li):  
         return sum(li)  
  
         sumOfListEle([10, 20, 30])
```

```
Out[33]: 60
```

07) Write a function to calculate the sum of the first element of each tuples inside the list.

```
In [34]: def sumOfFirstTupleEle(li):  
         return sum(tup[0] for tup in li)  
  
         sumOfFirstTupleEle([(10, 20, 30), (30, 20, 10), (20, 30, 10)])
```

```
Out[34]: 60
```

08) Write a recursive function to find nth term of Fibonacci Series.

```
In [59]: def fibonacci(n):  
         if n == 1:  
             return 0  
         if n == 2:  
             return 1
```

```
    return fibonacci(n-1) + fibonacci(n-2)

fibonacci(10)
```

Out[59]: 34

09) Write a function to get the name of the student based on the given rollno.

Example: Given dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'} find name of student whose rollno = 103

```
In [36]: dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'}

def rollname(rollN):
    return dict1[rollN]

rollname(103)
```

Out[36]: 'Jay'

10) Write a function to get the sum of the scores ending with zero.

Example : scores = [200, 456, 300, 100, 234, 678]

Ans = 200 + 300 + 100 = 600

```
In [37]: def lastzerosum(li):
    return sum(i for i in li if i % 10 == 0)

lastzerosum([200, 456, 300, 100, 234, 678])
```

Out[37]: 600

11) Write a function to invert a given Dictionary.

hint: keys to values & values to keys

Before : {'a': 10, 'b':20, 'c':30, 'd':40}

After : {10:'a', 20:'b', 30:'c', 40:'d'}

```
In [41]: def invertdict(dictionary):
    return {v: k for k, v in dictionary.items()}

dictt = {'a': 10, 'b':20, 'c':30, 'd':40}

invertdict(dictt)
```

Out[41]: {10: 'a', 20: 'b', 30: 'c', 40: 'd'}

12) Write a function that returns the number of uppercase and lowercase letters in the given string.

example : Input : s1 = AbcDEfgh ,Oupptput : no_upper = 3, no_lower = 5

```
In [46]: def uplowcasecounter(strr):  
         print("no_upper =", len([i for i in strr if i.isupper()]))  
         print("no_lower =", len([i for i in strr if i.islower()]))  
  
         uplowcasecounter('AbcDEfgh')
```

no_upper = 3

no_lower = 5

13) Write a lambda function to get smallest number from the given two numbers.

```
In [49]: (lambda a, b: a if a < b else b)(10, 20)
```

Out[49]: 10

14) For the given list of names of students, extract the names having more that 7 characters. Use filter().

```
In [52]: studentList = ["Yash", "Nilesh", "Devo", "Manav", "Bhavik", "Zeel", "Nirav", "Me  
         print(list(filter(lambda x: len(x) > 4, studentList)))
```

['Nilesh', 'Manav', 'Bhavik', 'Nirav']

15) For the given list of names of students, convert the first letter of all the names into uppercase. use map().

```
In [57]: studentList = ["yash", "nilesh", "devo", "manav", "bhavik", "zeel", "nirav", "me  
         print(list(map(lambda x: x.capitalize(), studentList)))
```

['Yash', 'Nilesh', 'Devo', 'Manav', 'Bhavik', 'Zeel', 'Nirav', 'Meet']

16) Write udfs to call the functions with following types of arguments:

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Legngth Positional(*args) & variable length Keyword Arguments (**kwargs)
5. Keyword-Only & Positional Only Arguments

```
In [60]: def posiArguFun(a, b, c):  
         print(a, b ,c)  
  
         posiArguFun(10, 20, 30)
```

10 20 30

```
In [61]: def kwArguFun(a, b, c):  
         print(a, b ,c)
```

```
kwArguFun(a=10, c=20, b=30)
```

```
10 30 20
```

```
In [64]: def defaultArguFun(a=10, b=20, c=30):  
         print(a, b, c)
```

```
defaultArguFun()  
defaultArguFun(a=20)  
defaultArguFun(b=50)  
defaultArguFun(a=40, c=50)  
defaultArguFun(a=40, b=80, c=50)
```

```
10 20 30  
20 20 30  
10 50 30  
40 20 50  
40 80 50
```

```
In [66]: def variLenPosiArguFun(*args):  
         print(args)
```

```
variLenPosiArguFun(10, 20, 30)
```

```
(10, 20, 30)
```

```
In [68]: def variLenKeyWordArguFun(**kwargs):  
         print(kwargs)
```

```
variLenKeyWordArguFun(a=10, b=20, c=30)
```

```
{'a': 10, 'b': 20, 'c': 30}
```

```
In [78]: def posiOnlyFun(a, b, c, /):  
         print(a, b, c)
```

```
posiOnlyFun(10, 20, 30)  
# posiOnlyFun(10, 20, c=30) TypeError: print() got an unexpected keyword argumen
```

```
10 20 30
```

```
In [82]: def keyWordOnlyFun(*, a, b, c):  
         print(a, b, c)
```

```
# keyWordOnlyFun(10, 20, 30) keyWordOnlyFun() takes 0 positional arguments but 3  
keyWordOnlyFun(a=10, b=20, c=30)
```

```
10 20 30
```

 No description has been provided for this image

Python Programming - 2301CS404

Lab - 10

Roll No : 412

Name : Narola Yash N.

01) WAP to read and display the contents of a text file. (also try to open the file in some other directory)

- in the form of a string
- line by line
- in the form of a list

```
In [21]: fp = open("Hello.txt", "r")
print(fp.read())
fp.close()
```

```
Hello! I am a file
This file is for demo
Do you like programming in Python?
Programming in python is really fun!
```

```
In [ ]:
```

```
In [22]: fp = open("Hello.txt")
print(fp.readline(), end="")
print(fp.readline(), end="")
fp.close()
```

```
Hello! I am a file
This file is for demo
```

```
In [23]: fp = open("Hello.txt")
print(fp.readlines())
fp.close()
```

```
['Hello! I am a file\n', 'This file is for demo\n', 'Do you like programming in P
ython?\n', 'Programming in python is really fun!\n']
```

```
In [24]: fp = open("../Extra Labs/AnoutherHello.txt")
print(fp.read(), end="")
fp.close()
```

Another Hello! Enjoy your python programming
Would you like tea or coffee ?
I like coffee with python!

02) WAP to create file named "new.txt" only if it doesn't exist.

```
In [25]: fp = open("new.txt", "w")
fp.write("Hello!\nHow You feel when you programming in python.\nIt's like mind b")
fp.close()
```

03) WAP to read first 5 lines from the text file.

```
In [34]: with open("Hello.txt", "r") as fp:
data = fp.readlines()

for i in range(5):
    print(data[i], end="")
```

Hello! I am a file
This file is for demo
Do you like programming in Python?
Programming in python is really fun!
This is line for...

04) WAP to find the longest word(s) in a file.

```
In [48]: with open("Hello.txt") as fp:
data = fp.read().split()
len_of_longest = len(max(data, key=len))

print([w for w in data if len_of_longest == len(w)])
```

['programming', 'Programming']

05) WAP to count the no. of lines, words and characters in a given text file.

```
In [53]: with open("Hello.txt") as fp:
print("No. of lines:", len(fp.readlines()))
fp.seek(0, 0)
print("No. of words:", len(fp.read().split()))
fp.seek(0, 0)
print("No. of character:", len(fp.read()))
```

No. of lines: 6
No. of words: 30
No. of character: 155

06) WAP to copy the content of a file to the another file.

```
In [56]: with open("Hello.txt") as fp:
          data = fp.read()

          with open("Hello_2.txt", "w") as fp:
              fp.write(data)

          with open("Hello_2.txt") as fp:
              print(fp.read())
```

Hello! I am a file
This file is for demo
Do you like programming in Python?
Programming in python is really fun!
This is line for...
I wanna play football.

07) WAP to find the size of the text file.

```
In [57]: with open("Hello_2.txt") as fp:
          print(len(fp.read()))
```

155

08) WAP to create an UDF named frequency to count occurrences of the specific word in a given text file.

```
In [59]: def ferquency(word, file):
          with open(file, "r") as fp:
              return (fp.read().lower()).count(word.lower())

          ferquency("Programming", "Hello.txt")
```

Out[59]: 2

09) WAP to get the score of five subjects from the user, store them in a file. Fetch those marks and find the highest score.

```
In [63]: with open("marks.txt", "w") as fp:
          for i in range(5):
              fp.write(input(f"Enter the marks of subject {i+1} :") + " ")

          with open("marks.txt") as fp:
              marks = fp.read().split()
              print(marks)
              print(max(marks))
```

['10', '20', '30', '40', '50']
50

10) WAP to write first 100 prime numbers to a file named primenumbers.txt

(Note: each number should be in new line)


```
In [79]: def is_prime(num):
    if num < 2:
        return False
    for i in range(2, num):
        if num % i == 0:
            return False
    return True

    with open("primenumber.txt", "w") as fp:
        count = 0
        num = 2
        while count < 100:
            if is_prime(num):
                fp.write(str(num) + '\n')
                count += 1
            num += 1
```

```
In [ ]: def is_prime(n):
    return n > 1 and all(n % i != 0 for i in range(2, n))

    with open("primenumbers.txt", "w") as f:
        n = 2
        c = 0
        while c < 100:
            if is_prime(n):
                f.write(f"{n}\n")
                c += 1
            n += 1
```

```
In [ ]: with open("primenumbers.txt", "w") as fp:
    count = 0
    num = 2

    while count < 100:
        if is_prime(num):
            fp.write(str(num) + "\n")    # each on new line
            count += 1
        num += 1

    print("First 100 prime numbers written to file.")
```

11) WAP to merge two files and write it in a new file.

```
In [82]: with open("Hello.txt") as fp:
    data1 = fp.read()

    with open("Hello_2.txt") as fp:
        data2 = fp.read()

    with open("newHello.txt", "w") as fp:
        fp.write(data1 + '\n' + data2)
```

12) WAP to replace word1 by word2 of a text file. Write the updated data to new file.

```
In [85]: with open("Hello.txt", "r") as fp:
          data = fp.read()

          with open("UpdatedHello.txt", "w") as fp:
              fp.write(data.replace("file", "folder"))
```

13) Demonstrate tell() and seek() for all the cases(seek from beginning-end-current position) taking a suitable example of your choice.

```
In [ ]: # Hello! I am a file
        # This file is for demo
        # Do you like programming in Python?
        # Programming in python is really fun!
        # This is line for...
        # I wanna play football.
```

```
In [112... with open("Hello.txt", "rb") as fp:
            print(fp.read(5).decode())
            print(fp.tell())
            print(fp.read(1).decode())
            print(fp.tell())
            fp.seek(0, 0)
            print(fp.read(6).decode())
            fp.seek(0, 2)
            print(fp.read().decode())
            fp.seek(0, 0)
            print(fp.read(7).decode())
            fp.seek(2, 1)
            print(fp.read(2).decode())
```

```
Hello
5
!
6
Hello!

Hello!
am
```

 No description has been provided for this image

Python Programming - 2301CS404

Lab - 11

Roll No : 412

Name : Narola Yash N.

Progrmas based on Python Data Structure + UDF + File Handling

Practical 1: Count Lines, Words and Characters

Problem Definition:

Write a Python program to read text from a file and count the total number of lines, words, and characters. The result should be written to an output file.

```
In [17]: with open("Hello.txt", "r") as fs:
          lines = len(fs.readlines())
          fs.seek(0, 0)
          word = len(fs.read().split())
          fs.seek(0, 0)
          character = len(fs.read())

          with open("output.txt", "a") as fs:
              fs.write(f"\nPractical 1: {lines} {word} {character}")
```

Practical 2: Search a Word in a File

Problem Definition:

Write a Python program to search a given word in a text file and count its occurrences. Store the result in an output file.

```
In [30]: with open("Hello.txt", "r") as fp:
          search = input("Which word you want to find:")
          words = fp.read().split()
          pure_words = [i.strip("!,.?").lower() for i in words]
          count = pure_words.count(search)
```

```
with open("output.txt", "a") as fs:
    fs.write(f"\nPractical 2: word: {search}, search count = {count}")
```

Practical 3: Student Marks and Average

Problem Definition:

Write a Python program to read roll number, name, and marks of students from a file. Store marks in a list, calculate the average using a user-defined function, and write student details along with average marks to an output file.

```
In [36]: def calculate_arg(marks):
        return sum([int(i) for i in marks]) / len(marks)

        with open("studentDetails.txt", "r") as fp:
            data = fp.read().split(", ")
            roll, name, marks = data[0], data[1], data[2].split()
            avg = calculate_arg(marks)

        with open("output.txt", "a") as fs:
            fs.write(f"\nPractical 3:\nStudent Roll NO: {roll}\nStudent Name: {name}\nSt
```

Practical 4: Maximum and Minimum using Tuple

Problem Definition:

Write a Python program to read numbers from a file, store them in a tuple, and find the maximum and minimum values. Write the result to an output file.

```
In [56]: with open("numbers.txt", "r") as fp:
        nums = tuple([int(i) for i in fp.read().split()])
        maxx = max(nums)
        minn = min(nums)

        with open("output.txt", "a") as fs:
            fs.write(f"\nPractical 4: max number: {maxx}\tmin number: {minn}")
```

Practical 5: Remove Duplicate Words using Set

Problem Definition:

Write a Python program to read words from a file and remove duplicate words using a set. Store the unique words in an output file.

```
In [57]: with open("Hello.txt", "r") as fs:
        unique = set([i.strip("!.?") for i in fs.read().split()])

        with open("output.txt", "a") as fs:
            fs.write(f"\nPractical 5: Unique words: { ", ".join(unique) }")
```

Practical 6: Word Frequency using Dictionary

Problem Definition:

Write a Python program to count the frequency of each word in a text file using a

dictionary and write the result to an output file.

```
In [58]: with open("Hello.txt", "r") as fs:
        pure_words = [i.strip("!,!?") for i in fs.read().split()]
        dictt = {}
        for k in pure_words:
            dictt[k.lower()] = dictt.get(k.lower(), 0) + 1

        with open("output.txt", "a") as fs:
            fs.write(f"\nPractical 6:result: {dictt}")
```

Practical 7: Student Result and Grade Calculation

Problem Definition:

Write a Python program to read roll number, name, and marks of students from a file.

Use a user-defined function to determine grade (A/B/C/Fail) and write the student result to an output file.

Grade Criteria:

- Marks ≥ 75 : Grade A
- Marks ≥ 60 : Grade B
- Marks ≥ 40 : Grade C
- Marks < 40 : Fail

```
In [80]: def cal_grede(marks):
        avg = sum(marks) / len(marks)

        if avg >= 75:
            return 'A'
        elif avg >= 60:
            return 'B'
        elif avg >= 40:
            return 'C'
        else:
            return 'Fail'

        with open("studentDetails.txt", "r") as fp:
            data = [tuple(i.split(", ")) for i in fp.read().split("\n")]
            modified_data = []
            for i in data:
                i += tuple(cal_grede([int(j) for j in i[2].split()]))
                modified_data.append(i)

            print(modified_data)

        with open("output.txt", "a") as fs:
            fs.write(f"\nPractical 7:\n {modified_data}")
```

```
[('413', 'Nilesh', '98 99 97 95 92', 'A'), ('412', 'Yash', '12 50 63 40 10', 'F',
'a', 'i', 'l'), ('418', 'Bhavik', '80 40 50 87 08', 'C'), ('419', 'Devanshu', '98
74 85 85 78', 'A')]
```

```
In [ ]:
```


 No description has been provided for this image

Python Programming - 2301CS404

Lab - 12

Roll No : 412

Name : Narola Yash N.

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [12]: try:
          a = int(input("Enter the first value:"))
          b = int(input("Enter the second value:"))
          print("Your numbers is " + str(a) + ', ' + str(b))
          print(a // b)
        except ZeroDivisionError as err:
          print("Error type:", type(err))
          print("Error message:", err)
```

```
Your numbers is 1, 0
Error type: <class 'ZeroDivisionError'>
Error message: integer division or modulo by zero
```

```
In [14]: try:
          a = int(input("Enter the first value:"))
          b = int(input("Enter the second value:"))
          print("Your numbers is " + str(a) + ', ' + str(b))
          print(a // b)
        except TypeError as err:
          print("Error type:", type(err))
          print("Error message:", err)
```

Error type: <class 'TypeError'>
Error message: can only concatenate str (not "int") to str

```
In [16]: try:
        a = int(input("Enter the first value:"))
        b = int(input("Enter the second value:"))
        print("Your numbers is " + str(a) + ', ' + str(b))
        print(a // b)
    except ValueError as err:
        print("Error type:", type(err))
        print("Error message:", err)
```

Error type: <class 'ValueError'>
Error message: invalid literal for int() with base 10: 'abc'

```
In [17]: try:
        a = int(input("Enter the first value:"))
        b = int(input("Enter the second value:"))
        print("Your numbers is " + str(a) + ', ' + str(b))
        print(a // b)
    except Exception as err:
        print("Error type:", type(err))
        print("Error message:", err)
```

Your numbers is 1, 0
Error type: <class 'ZeroDivisionError'>
Error message: integer division or modulo by zero

```
In [18]: try:
        a = int(input("Enter the first value:"))
        b = int(input("Enter the second value:"))
        print("Your numbers is " + str(a) + ', ' + str(b))
        print(a // b)
    except Exception as err:
        print("Error type:", type(err))
        print("Error message:", err)
```

Error type: <class 'ValueError'>
Error message: invalid literal for int() with base 10: 'a'

```
In [19]: try:
        a = int(input("Enter the first value:"))
        b = int(input("Enter the second value:"))
        print("Your numbers is " + str(a) + ', ' + b)
        print(a // b)
    except Exception as err:
        print("Error type:", type(err))
        print("Error message:", err)
```

Error type: <class 'TypeError'>
Error message: can only concatenate str (not "int") to str

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

```
In [22]: listt = [1, 2, 3, 4, 5]
        print(listt[1])
        print(listt[10])
```


2

```
-----
IndexError                                Traceback (most recent call last)
Cell In[22], line 3
      1 listt = [1, 2, 3, 4, 5]
      2 print(listt[1])
----> 3 print(listt[10])

IndexError: list index out of range
```

```
In [27]: try:
          listt = [1, 2, 3, 4, 5]
          print(listt[1])
          print(listt[10])
        except IndexError as err:
          print("Error type:", type(err))
          print("Error message:", err)
```

2

Error type: <class 'IndexError'>
Error message: list index out of range

```
In [30]: dicct = {1: "One", 2: "Two"}
          print(dicct[1])
          print(dicct[3])
```

One

```
-----
KeyError                                Traceback (most recent call last)
Cell In[30], line 3
      1 dicct = {1: "One", 2: "Two"}
      2 print(dicct[1])
----> 3 print(dicct[3])

KeyError: 3
```

```
In [31]: try:
          dicct = {1: "One", 2: "Two"}
          print(dicct[1])
          print(dicct[3])
        except KeyError as err:
          print("Error type:", type(err))
          print("Error message:", err)
```

One

Error type: <class 'KeyError'>
Error message: 3

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```
In [32]: fp = open("xyx.txt", "r")
          fp.read()
```

```

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[32], line 1
----> 1 fp = open("xyx.txt", "r")
      2 fp.read()

File ~\anaconda3\Lib\site-packages\IPython\core\interactiveshell.py:324, in _modified_open(file, *args, **kwargs)
    317 if file in {0, 1, 2}:
    318     raise ValueError(
    319         f"IPython won't let you open fd={file} by default "
    320         "as it is likely to crash IPython. If you know what you are doing, "
    321         "you can use builtins' open."
    322     )
--> 324 return io_open(file, *args, **kwargs)

FileNotFoundError: [Errno 2] No such file or directory: 'xyx.txt'

```

```

In [33]: try:
        fp = open("xyx.txt", "r")
        fp.read()
    except FileNotFoundError as err:
        print("Error type:", type(err))
        print("Error message:", err)

```

Error type: <class 'FileNotFoundError'>
 Error message: [Errno 2] No such file or directory: 'xyx.txt'

```

In [34]: import xyz

```

```

-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[34], line 1
----> 1 import xyz

ModuleNotFoundError: No module named 'xyz'

```

```

In [35]: try:
        import xyz
    except ModuleNotFoundError as err:
        print("Error type:", type(err))
        print("Error message:", err)

```

Error type: <class 'ModuleNotFoundError'>
 Error message: No module named 'xyz'

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```

In [44]: try:
        a = int(input("Enter the first value:"))
        b = int(input("Enter the second value:"))
        print("Your numbers is " + str(a) + ', ' + str(b))
        # print("Your numbers is " + str(a) + ', ' + str(b))
        print(a // b)

        # listt = [1, 2, 3, 4, 5]
        # print(listt[1])
    except:
        print("Error: Invalid input or operation")

```

```

# print(listt[10])

# dicct = {1: "One", 2: "Two"}
# print(dicct[1])
# print(dicct[3])

# fp = open("xyx.txt", "r")
# fp.read()

# import xyz
except Exception as exce:
    print("Error type:", type(exce))
    print("Error message:", exce)

```

Error type: <class 'TypeError'>

Error message: can only concatenate str (not "int") to str

05) WAP to demonstrate else and finally block.

```

In [50]: try:
        age = int(input("Enter your age:"))
        if age <= 0:
            raise ValueError("Age can not be less then or equal zero")
        except Exception as exce:
            print("Error type:", type(exce))
            print("Error message:", exce)
        else:
            print("Your age is", age)
        finally:
            print("Exit")

```

Your age is 18

Exit

```

In [51]: try:
        age = int(input("Enter your age:"))
        if age <= 0:
            raise ValueError("Age can not be less then or equal zero")
        except Exception as exce:
            print("Error type:", type(exce))
            print("Error message:", exce)
        else:
            print("Your age is", age)
        finally:
            print("Exit")

```

Error type: <class 'ValueError'>

Error message: Age can not be less then or equal zero

Exit

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```

In [58]: def divide():
        try:
            a = int(input("Enter the first value:"))
            b = int(input("Enter the second value:"))
            result = a // b
        except ZeroDivisionError as exce:

```

```

        print("Error type:", type(exce))
        print("Error message:", exce)
    else:
        return result

d = divide()
print("xyz", d)

```

Error type: <class 'ZeroDivisionError'>
 Error message: integer division or modulo by zero
 xyz None

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

```

In [62]: def billing(item, price):
        try:
            if price < 0:
                raise ValueError("Price can not be negative")
        except ValueError as exce:
            print("Error type:", type(exce))
            print("Error message:", exce)
        else:
            bill = item * price
            print(bill)
        finally:
            print("Thank You For Visiting!")

# billing(10, 10)
billing(10, -10)

```

Error type: <class 'ValueError'>
 Error message: Price can not be negative
 Thank You For Visiting!

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```

In [84]: try:
        grede = [int(i) for i in input("Enter the gredes separated by commas").split]
    except Exception as exce:
        print("Error type:", type(exce))
        print("Error message:", exce)
    else:
        print(grede)

```

Error type: <class 'ValueError'>
 Error message: invalid literal for int() with base 10: 'a'

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```
In [74]: try:
marks = []
for i in range(5):
    markInOne = int(input("Enter the marks:"))
    assert 0 <= markInOne <= 100, 'Marks should be between 0 to 100'
    marks.append(markInOne)
except Exception as exce:
    print("Error type:", type(exce))
    print("Error message:", exce)
else:
    print("Valid marks entered")
    print("Average marks:", sum(marks) / len(marks))
finally:
    print("Exit")
```

Error type: <class 'AssertionError'>
Error message: Marks should be between 0 to 100
Exit

```
In [75]: try:
marks = []
for i in range(5):
    markInOne = int(input("Enter the marks:"))
    assert 0 <= markInOne <= 100, 'Marks should be between 0 to 100'
    marks.append(markInOne)
except Exception as exce:
    print("Error type:", type(exce))
    print("Error message:", exce)
else:
    print("Valid marks entered")
    print("Average marks:", sum(marks) / len(marks))
finally:
    print("Exit")
```

Valid marks entered
Average marks: 70.4
Exit

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```
In [70]: class WeakPasswordError(Exception):
        pass

        try:
            password = input("Enter your password:")
            if len(password) < 8 or not any(i.isdigit() for i in password):
                raise WeakPasswordError("Password is not valid")
        except Exception as exce:
            print("Error type:", type(exce))
            print("Error message:", exce)
        else:
            print("Correct")
        finally:
            print("Exit")
```

Error type: <class '__main__.WeakPasswordError'>

Error message: Password is not valid

Exit

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```
In [65]: class NegativeNumberError(Exception):
        pass

        try:
            num = int(input("Enter the number:"))
            if num < 0:
                raise NegativeNumberError("Cannot calculate the square roor of a negativ
        except NegativeNumberError as exce:
            print("Error type:", type(exce))
            print("Error message:", exce)
        else:
            print(num ** (1 / 2))
        finally:
            print("Exit")
```

5.0

Exit

```
In [66]: class NegativeNumberError(Exception):
        pass

        try:
            num = int(input("Enter the number:"))
            if num < 0:
```

```
        raise NegativeNumberError("Cannot calculate the square roor of a negativ
except NegativeNumberError as exce:
    print("Error type:", type(exce))
    print("Error message:", exce)
else:
    print(num ** (1 / 2))
finally:
    print("Exit")
```

Error type: <class '__main__.NegativeNumberError'>

Error message: Cannot calculate the square roor of a negative number

Exit

 No description has been provided for this image

Python Programming - 2301CS404

Lab - 13

Roll No.: 412

Name: Narola Yash N.

01) WAP to create Calculator module which defines functions like add, sub, mul and div.

Create another .py file that uses the functions available in Calculator module.

```
In [1]: %%writefile calculator.py
```

```
def add(a, b):  
    return a + b  
  
def sub(a, b):  
    return a - b  
  
def mul(a, b):  
    return a * b  
  
def div(a, b):  
    return a / b  
  
def floarDiv(a, b):  
    return a // b  
  
def mod(a, b):  
    return a % b
```

Writing calculator.py

```
In [2]: import calculator
```

```
In [4]: calculator.add(1, 2)
```

```
Out[4]: 3
```

```
In [5]: calculator.sub(1, 2)
```


Out[5]: -1

02) WAP to pick a random character from a given String.

```
In [ ]: import random
```

```
In [876... str_ = "This is String."  
random.choice(str_)
```

Out[876... 's'

03) WAP to pick a random element from a given list.

```
In [911... list_ = ["Yash", "Narola", 19, 8.5, 412]  
random.choice(list_)
```

Out[911... 19

04) WAP to roll a dice in such a way that every time you get the same number.

```
In [941... dice = [1, 2, 3, 4, 5, 6]  
  
random.seed(412)  
random.choice(dice)
```

Out[941... 3

05) WAP to generate 3 random integers between 100 and 999 which is divisible by 5.

```
In [943... list_ = [i for i in range(100, 1000) if i % 5 == 0]  
random.sample(list_, 3)
```

Out[943... [170, 235, 350]

06) WAP to generate 100 random lottery tickets and pick two lucky tickets from it and announce them as Winner and Runner up respectively.

```
In [978... random.seed(2)  
lottery_numbers = [random.randint(10001, 100000) for i in range(100)]  
  
winner, runnerUp = random.sample(lottery_numbers, 2)  
  
print("winner =", winner, "runnerUp =", runnerUp)
```

winner = 52555 runnerUp = 37816

07) WAP to print current date and time in Python.

```
In [1]: import datetime

now_ = datetime.datetime.now()

print(now_)
```

2026-03-11 11:00:38.938861

08) Subtract a week (7 days) from a given date in Python.

```
In [5]: date_ = datetime.datetime(2025, 4, 10)

past_date = date_ + datetime.timedelta(weeks = -1)
print(past_date)
```

2025-04-03 00:00:00

09) WAP to Calculate number of days between two given dates.

```
In [11]: delta_ = datetime.datetime.now() - datetime.datetime(2007, 3, 10)
print(delta_.days)
```

6941

10) WAP to Find the day of the week of a given date.(i.e. whether it is sunday/monday/tuesday/etc.)

```
In [13]: datee = datetime.date.today()
datetime.datetime.strftime(datee, "%A")
```

Out[13]: 'Wednesday'



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 14

Roll No.: 412

Name: Narola Yash N.

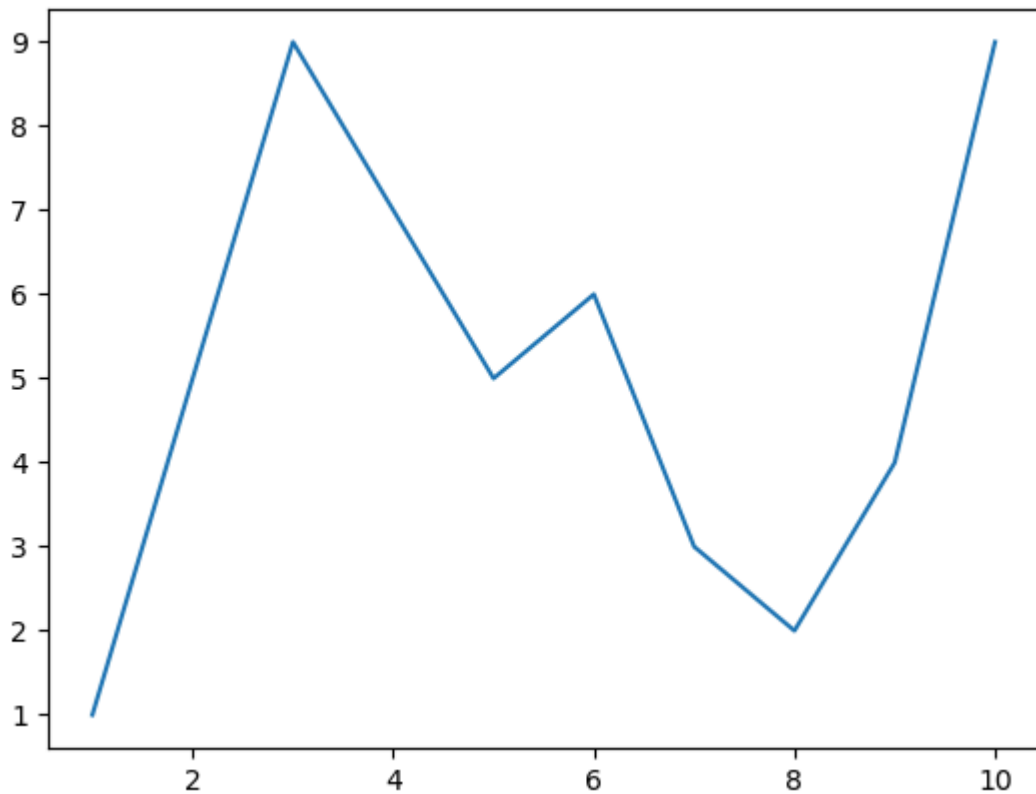
Import matplotlib below

```
In [2]: import matplotlib.pyplot as plt
```

01) WAP to display the line chart for given data for x and y axis.

```
In [2]: x = range(1,11)  
y = [1,5,9,7,5,6,3,2,4,9]
```

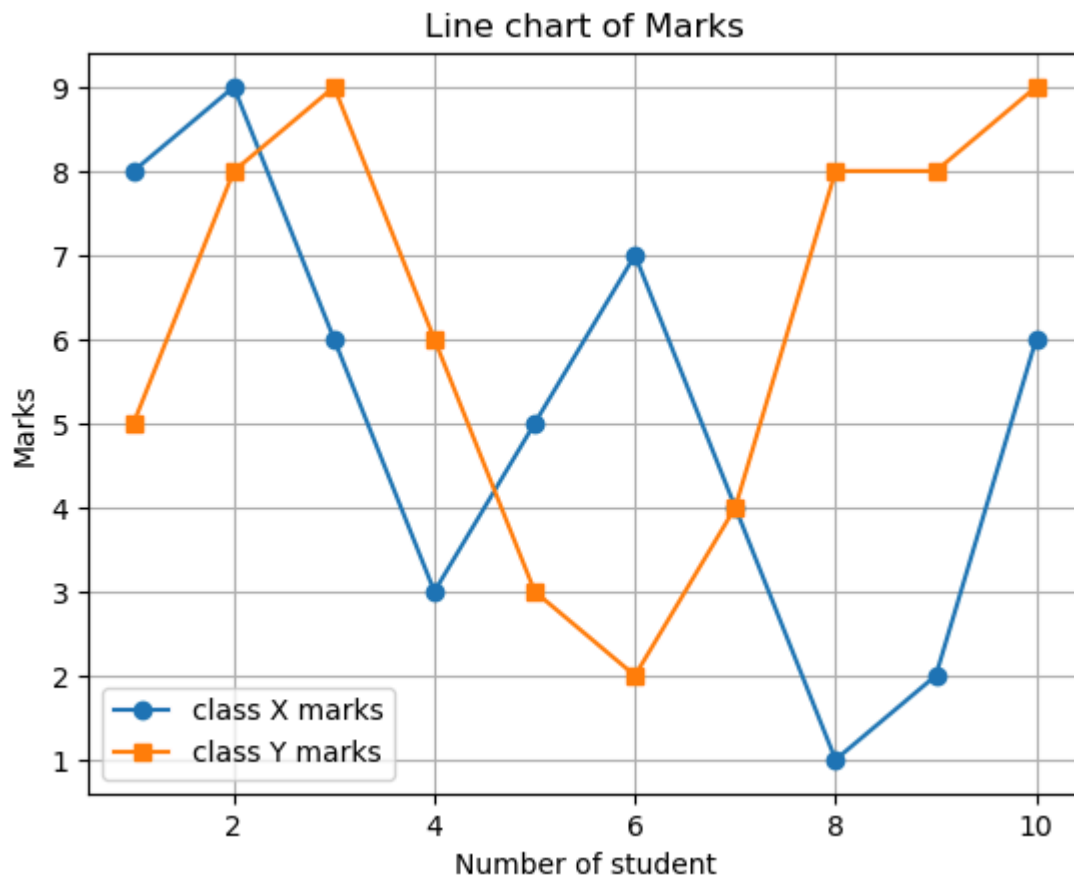
```
In [3]: plt.plot(x, y)  
plt.show()
```



02) WAP to generate line chart for given data.

```
In [4]: x = range(1,11,1)
cxMarks= [8,9,6,3,5,7,4,1,2,6]
cyMarks= [5,8,9,6,3,2,4,8,8,9]
```

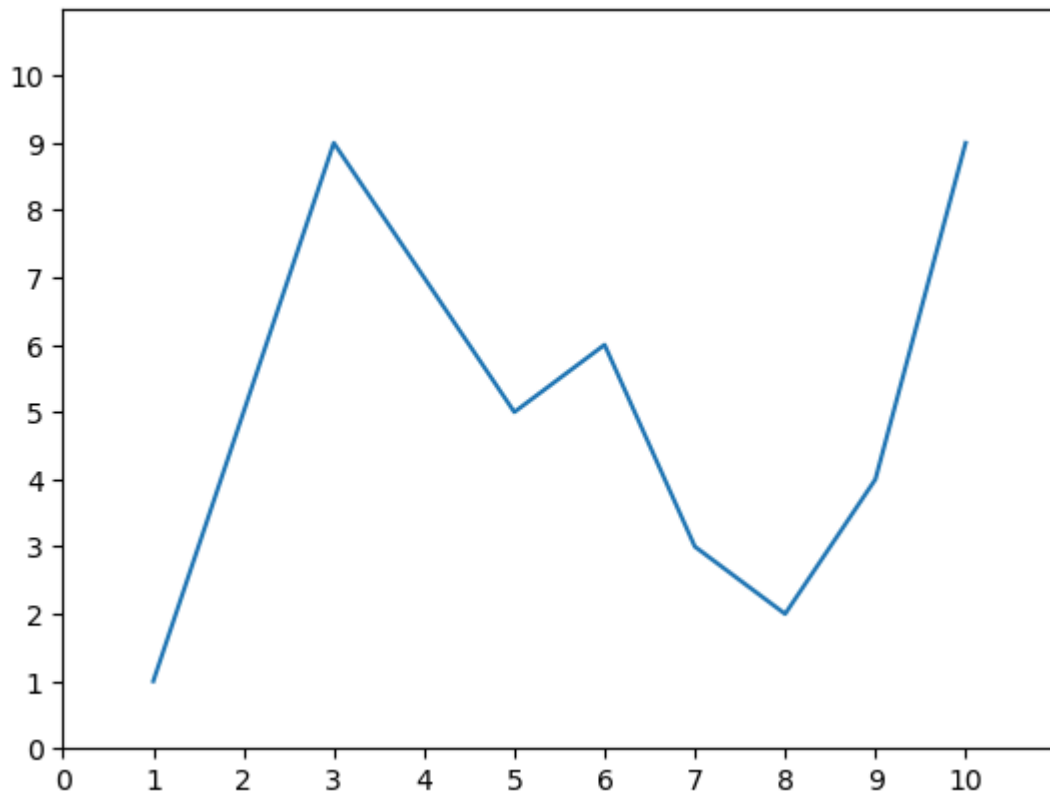
```
In [14]: plt.plot(x, cxMarks, marker='o', label='class X marks')
plt.plot(x, cyMarks, marker='s', label='class Y marks')
plt.title('Line chart of Marks')
plt.xlabel('Number of student')
plt.ylabel('Marks')
plt.grid(True)
plt.legend()
plt.show()
```



03) WAP to generate line chart for given data, but create your custom axis.

```
In [16]: x = range(1,11)
y = [1,5,9,7,5,6,3,2,4,9]
```

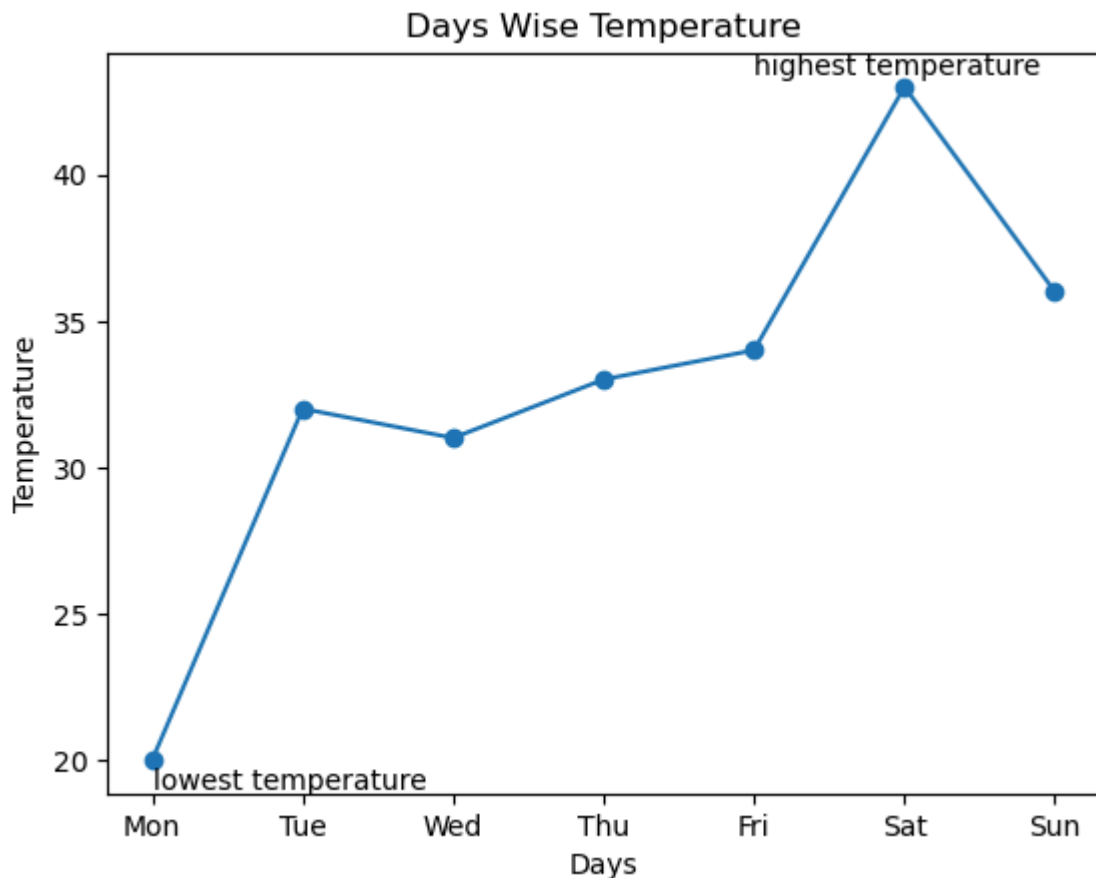
```
In [19]: ax = plt.axes()
ax.set_xlim([0,11])
ax.set_ylim([0,11])
ax.set_xticks(range(0,11))
ax.set_yticks(range(0,11))
plt.plot(x, y)
plt.show()
```



04) WAP to plot a line chart showing temperature variation during a week. Mark the lowest and highest temperature.

```
In [21]: Days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]  
Temperature = [20, 32, 31, 33, 34, 43, 36]
```

```
In [28]: plt.plot(Days, Temperature, marker='o')  
plt.xlabel("Days")  
plt.ylabel("Temperature")  
plt.title("Days Wise Temperature")  
plt.annotate("lowest temperature", xy = ["Mon", 19])  
plt.annotate("highest temperature", xy = ["Fri", 43.4])  
plt.show()
```



05) WAP to create a detailed pie chart representing the monthly expenditure distribution of a family.

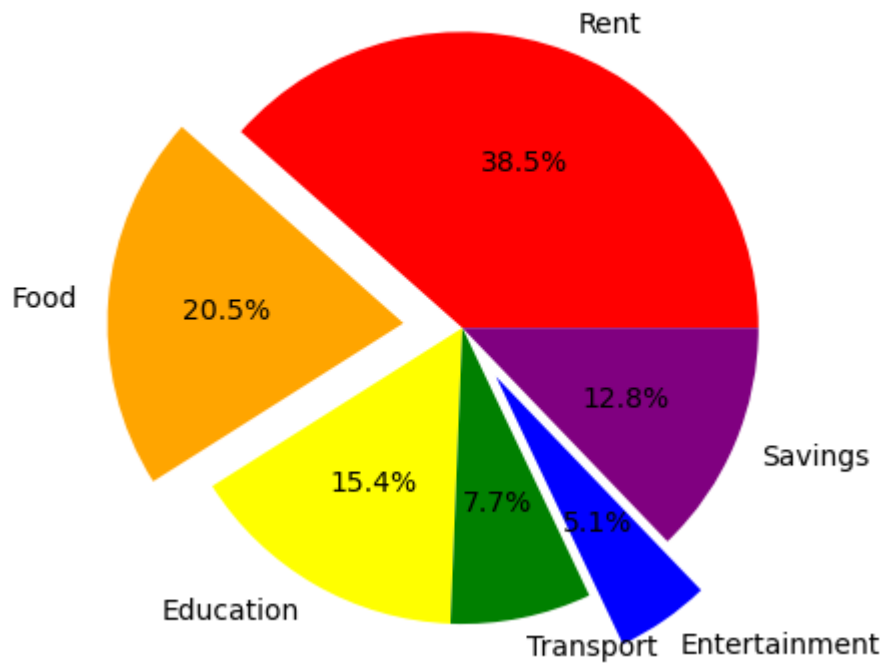
The chart should display labels, percentage values, title, different colors for each slice, and explode one slice to highlight the highest expense.

```
In [ ]: Categories = ["Rent", "Food", "Education", "Transport", "Entertainment", "Saving"]
Amount (₹) = [15000, 8000, 6000, 3000, 2000, 5000]
```

```
In [45]: Categories = ["Rent", "Food", "Education", "Transport", "Entertainment", "Saving"]
Amount = [15000, 8000, 6000, 3000, 2000, 5000]

plt.pie(Amount, labels=Categories, autopct='%1.1f%%', colors=['red', 'orange', 'yellow', 'green', 'blue', 'purple'])
plt.title("Monthly Expenditure Distribution of A Family")
plt.show()
```

Monthly Expenditure Distribution of A Family

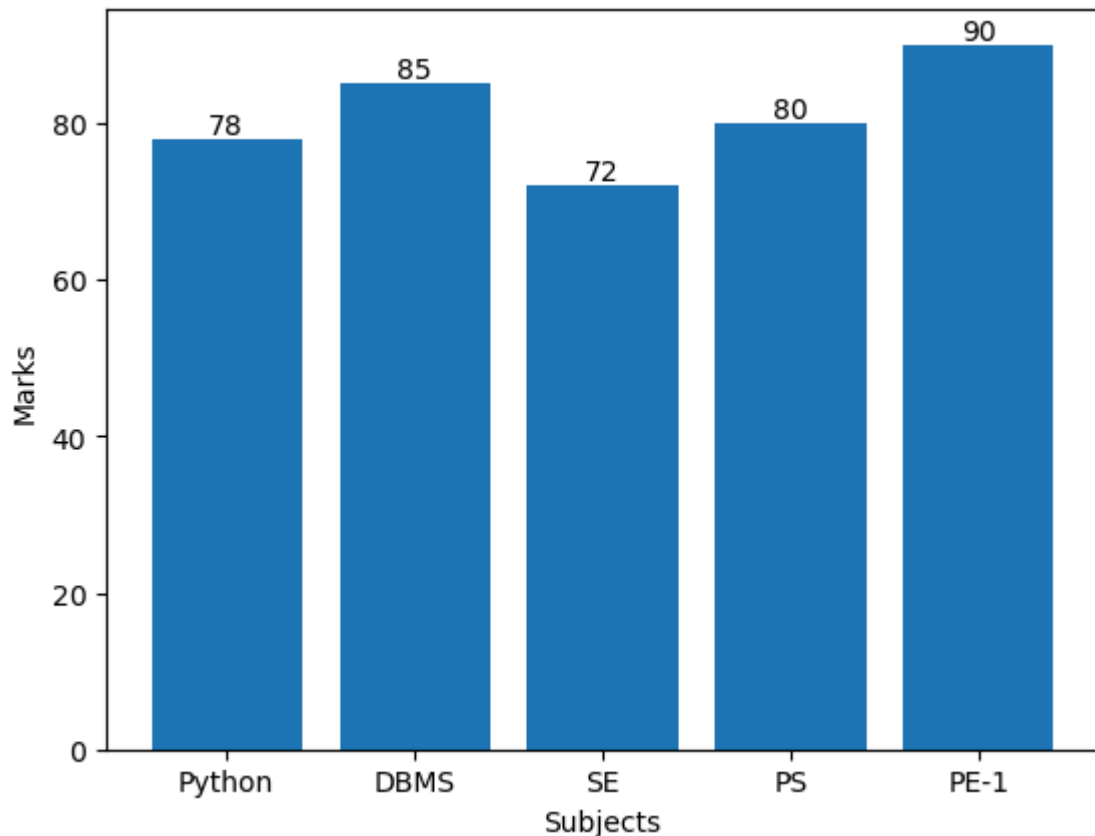


06) WAP to create bar chart showing the marks obtained by students in different subjects.

The program should also display the marks on the top of the respective bar.

```
In [5]: Subjects = ["Python", "DBMS", "SE", "PS", "PE-1"]  
Marks = [78, 85, 72, 80, 90]
```

```
In [6]: bars = plt.bar(Subjects, Marks)  
plt.xlabel("Subjects")  
plt.ylabel("Marks")  
plt.bar_label(bars)  
plt.show()
```

07) WAP to create a horizontal bar chart representing the number of books sold in different categories.

The program should also display the number of books sold at the end of the respective bar.

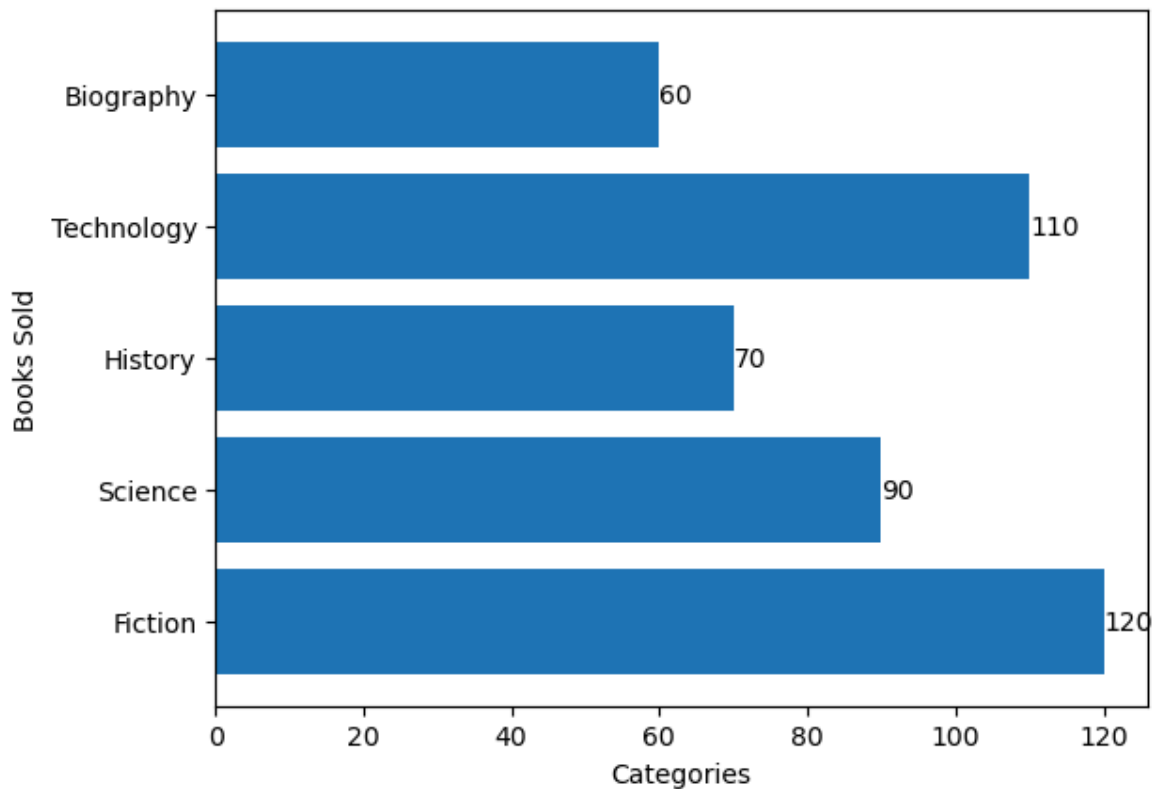
```
In [8]: Categories = ["Fiction", "Science", "History", "Technology", "Biography"]
Books Sold = [120, 90, 70, 110, 60]
```

```
Cell In[8], line 2
    Books Sold = [120, 90, 70, 110, 60]
    ^
```

SyntaxError: invalid syntax

```
In [9]: Categories = ["Fiction", "Science", "History", "Technology", "Biography"]
Books_Sold = [120, 90, 70, 110, 60]

bars = plt.barh(Categories, Books_Sold)
plt.xlabel("Categories")
plt.ylabel("Books Sold")
plt.bar_label(bars)
plt.show()
```

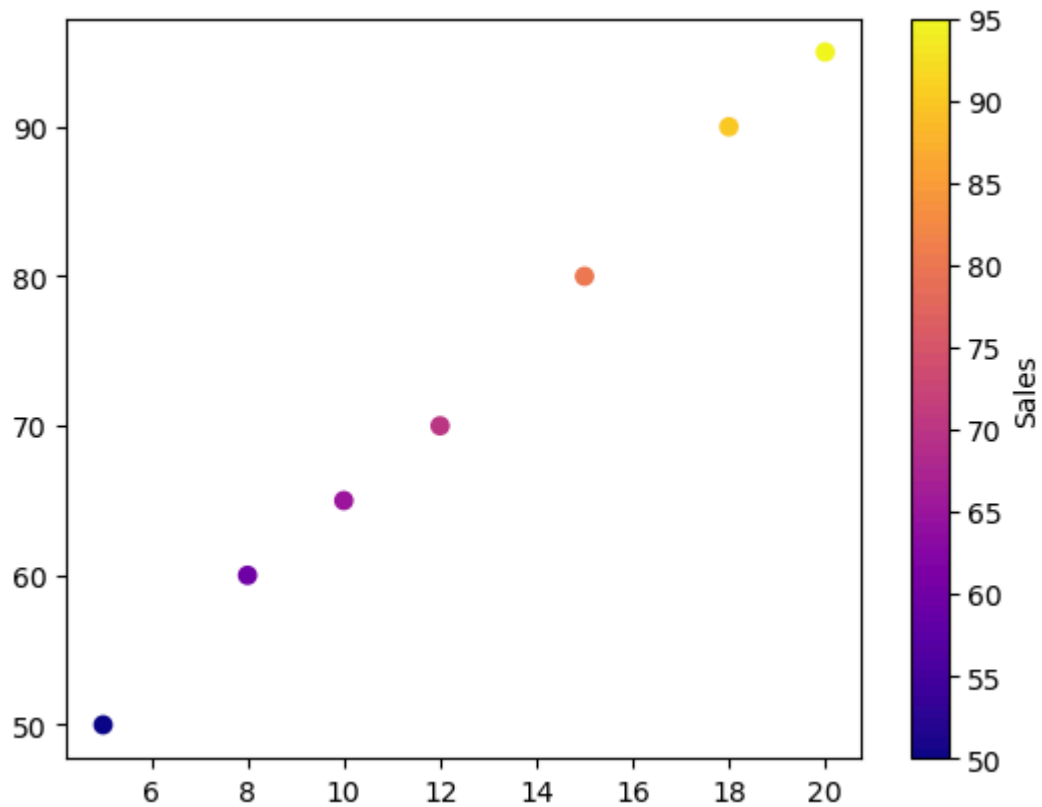


08) WAP to create a scatter plot representing the relationship between advertising expenditure and sales revenue of a company.

```
In [ ]: Advertising Cost (₹ thousands) = [5, 8, 10, 12, 15, 18, 20]
Sales Revenue (₹ thousands) = [50, 60, 65, 70, 80, 90, 95]
```

```
In [10]: Advertising_Cost = [5, 8, 10, 12, 15, 18, 20]
Sales_Revenue = [50, 60, 65, 70, 80, 90, 95]

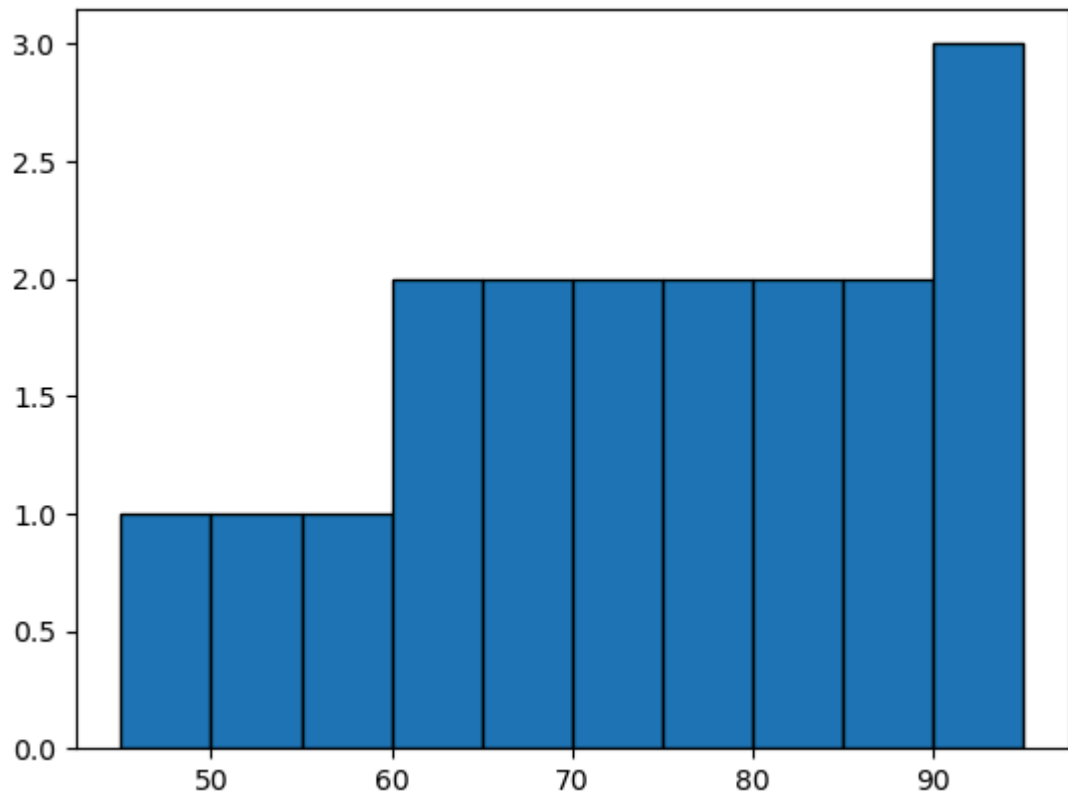
plt.scatter(Advertising_Cost, Sales_Revenue, c = Sales_Revenue, cmap="plasma")
plt.colorbar(label = "Sales")
plt.show()
```



09) WAP to create a histogram representing the distribution of marks obtained by students in a class.

```
In [ ]: Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]
```

```
In [11]: Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]  
plt.hist(Marks,edgecolor="k")  
plt.show()
```

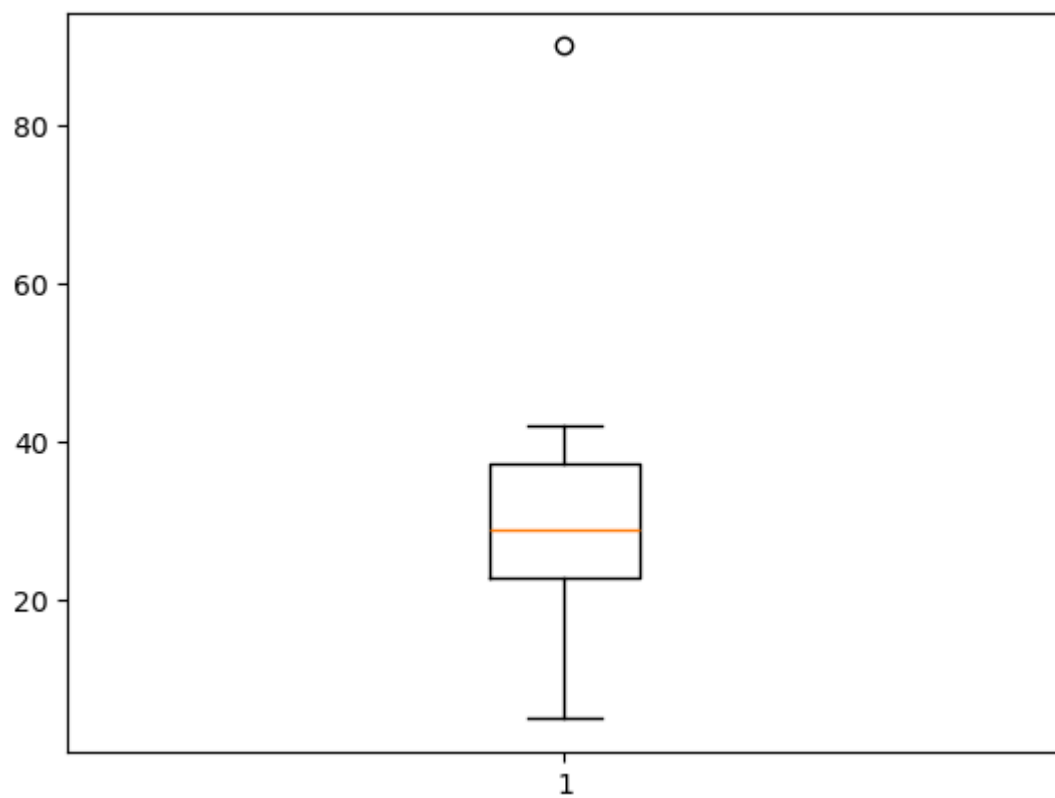


10) WAP to create a box plot representing the distribution of daily commuting time (in minutes) of employees and identify outliers (if any).

```
In [ ]: Commuting Time (minutes) = [5, 18, 20, 22, 25, 27, 28, 30, 32, 35, 38, 40, 42, 90]
```

```
In [12]: Commuting_Time = [5, 18, 20, 22, 25, 27, 28, 30, 32, 35, 38, 40, 42, 90]

plt.boxplot(Commuting_Time)
plt.show()
```





Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 412

Name: Narola Yash N.

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

```
In [2]: class Students:
        def __init__(self, name, age, grade):
            self.name = name
            self.age = age
            self.grade = grade

        def __str__(self):
            return self.name
```

```
In [5]: s1 = Students("Yash", 18, "A++")
        print(s1)
```

Yash

02) Create a class named Bank_Account with Account_No, User_Name, Email, Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

```
In [9]: class Bank_Account:
        def GetAccountDetails(self):
            Account_No, User_Name, Email, Account_Type, Account_Balance = input(" Enter Account No, User Name, Email, Account Type and Account Balance: ")
            self.Account_No = int(Account_No)
            self.User_Name = User_Name
            self.Email = Email
            self.Account_Type = Account_Type
            self.Account_Balance = int(Account_Balance)
```

```

def DisplayAccountDetails(self):
    print("Account_No = ",self.Account_No)
    print("User_Name = ",self.User_Name)
    print("Email = ",self.Email)
    print("Account_Type = ",self.Account_Type)
    print("Account_Balance = ",self.Account_Balance)

b1 = Bank_Account()
b1.GetAccountDetails()
b1.DisplayAccountDetails()

```

```

Account_No = 412
User_Name = Yash
Email = yash@narola.com
Account_Type = fullsavings
Account_Balance = 5

```

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

```

In [10]: class Circle:
def __init__(self, radius):
    self.r = radius

def area(self):
    # return 3.14 * (self.r**2)
    print(3.14 * self.r * self.r)

def perimeter(self):
    print(2 * 3.14 * self.r)

c1 = Circle(5)
c1.area()
c1.perimeter()

```

```

78.5
31.400000000000002

```

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

```

In [11]: class employees:
def __init__(self, name, age, salary):
    self.name = name
    self.age = age
    self.salary = salary

def updateEmployeeDetails(self):
    name, age, salary = input(" Enter new name, age, salary:").split()
    self.name = name
    self.age = age
    self.salary = salary

def displayEmployeeDetails(self):
    print("name = ",self.name)

```

```

        print("age = ",self.age)
        print("salary = ",self.salary)

e1 = employees("Yash", 19, 12000)
e1.displayEmployeeDetails()
e1.updateEmployeeDetails()
e1.displayEmployeeDetails()

```

```

name = Yash
age = 19
salary = 12000
name = Yash
age = 18
salary = 50

```

05) Create a bank account class with methods to deposit, withdraw, and check balance.

```

In [12]: class bank_account:
        def __init__(self, amount):
            self.amount = amount

        def deposit(self, money):
            self.amount += money
            print(money, "added")

        def withdraw(self, money):
            if self.amount - money < 0:
                print("insufficient balance")
                return
            self.amount -= money
            print(money, "withdrawed")

        def checkBalance(self):
            print("balance = ", self.amount)

b1 = bank_account(5000)
b1.deposit(1000)
b1.withdraw(2000)
b1.checkBalance()

```

```

1000 added
2000 withdrawn
balance = 4000

```

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

```

In [13]: class Inventory:
        def __init__(self):
            self.items = {}

        def add_item(self, name, price, quantity):
            self.items[name] = [price, quantity]
            print("Item added")

        def remove_item(self, name):

```



```

        if name in self.items:
            del self.items[name]
            print("Item removed")
        else:
            print("Item not found")

    def update_item(self, name, price, quantity):
        if name in self.items:
            self.items[name] = [price, quantity]
            print("Item updated")
        else:
            print("Item not found")

    def show_items(self):
        for name in self.items:
            print(name, "Price:", self.items[name][0], "Quantity:", self.items[name][1])

inv = Inventory()

inv.add_item("Pen", 10, 50)
inv.add_item("Book", 100, 20)

inv.show_items()

inv.update_item("Pen", 12, 60)
inv.remove_item("Book")

inv.show_items()

```

```

Item added
Item added
Pen Price: 10 Quantity: 50
Book Price: 100 Quantity: 20
Item updated
Item removed
Pen Price: 12 Quantity: 60

```

07) Create a Class with instance attributes of your choice.

```

In [19]: class InstanceAttr:
        def __init__(self, name):
            self.name = name

        i1 = InstanceAttr("Nilesh")

        print(i1.name)

```

Nilesh

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

```
In [20]: class student_kit:
    principal_name = "Mr ABC"

    def __init__(self, name):
        self.name = name

    def attendance(self, days):
        print("Student Name:", self.name)
        print("Principal:", student_kit.principal_name)
        print("Days Present:", days)
        print("Certificate Generated")

name = input("Enter student name: ")
s1 = student_kit(name)

days = int(input("Enter number of days present: "))
s1.attendance(days)
```

Student Name: Nilesh
Principal: Mr ABC
Days Present: 5
Certificate Generated

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

```
In [21]: class Time:
    def __init__(self, hour, minute):
        self.hour = hour
        self.minute = minute

    def add(self, t1, t2):
        self.minute = t1.minute + t2.minute
        self.hour = t1.hour + t2.hour

        if self.minute >= 60:
            self.hour += 1
            self.minute -= 60

        print("Added Time:", self.hour, "hours", self.minute, "minutes")

t1 = Time(2, 40)
t2 = Time(1, 30)
t3 = Time(0, 0)

t3.add(t1, t2)
```

Added Time: 4 hours 10 minutes

In []:



Darshan
UNIVERSITY

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.: 412

Name: Narola Yash N.

Continued..

10) Calculate area of a rectangle using object as an argument to a method.

```
In [1]: class Rectangle:
        def __init__(self, length, width):
            self.length = length
            self.width = width

        def calculate_area(rect):
            return rect.length * rect.width

        r = Rectangle(10, 5)

        area = calculate_area(r)

        print("Area of Rectangle:", area)
```

Area of Rectangle: 50

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

```
In [9]: class Square:
        def __init__(self, side):
            self.side = side    # Constructor
```

```

def area(self):
    area_value = self.side * self.side
    self.output(area_value)    # Calling output() method

def output(self, area_value):
    print("Side of square:", self.side)
    print("Area of square:", area_value)

sq = Square(4)

sq.area()

```

Side of square: 4
Area of square: 16

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named `area()` and a method named `output()` that prints the output and is invoked by `area()`.

Also define a class method that compares the two sides of rectangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : **THIS IS SQUARE**.

```

In [ ]: class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        result = self.length * self.width
        self.output(result)    # Calling output method

    def output(self, result):
        print("Length:", self.length)
        print("Width:", self.width)
        print("Area of Rectangle:", result)

    @classmethod
    def create(cls, length, width):
        if length == width:
            print("THIS IS SQUARE")
            return None
        else:
            return cls(length, width)

rect = Rectangle.create(5, 3)

if rect:
    rect.area()

```

13) Define a class Square having a private attribute "side".

Implement `get_side` and `set_side` methods to access the private attribute from outside of the class.

```
In [10]: class Square:
    def __init__(self, side):
        self.__side = side  # private attribute

    # Getter method
    def get_side(self):
        return self.__side

    # Setter method
    def set_side(self, side):
        if side > 0:
            self.__side = side
        else:
            print("Side must be positive!")

    # Example usage
    s = Square(5)

    print("Initial side:", s.get_side())

    s.set_side(10)
    print("Updated side:", s.get_side())

    s.set_side(-3)  # Invalid input
```

```
Initial side: 5
Updated side: 10
Side must be positive!
```

14) Create a class `Profit` that has a method named `getProfit` that accepts profit from the user.

Create a class `Loss` that has a method named `getLoss` that accepts loss from the user.

Create a class `BalanceSheet` that inherits from both classes `Profit` and `Loss` and calculates the balance. It has two methods `getBalance()` and `printBalance()`.

```
In [1]: class Profit:
    def getProfit(self):
        self.profit = float(input("Enter profit: "))

class Loss:
    def getLoss(self):
        self.loss = float(input("Enter loss: "))

class BalanceSheet(Profit, Loss):
    def getBalance(self):
        self.balance = self.profit - self.loss
```

```

    def printBalance(self):
        print("Balance:", self.balance)

# Example usage
b = BalanceSheet()

b.getProfit()
b.getLoss()
b.getBalance()
b.printBalance()

```

Balance: 95000.0

15) WAP to demonstrate all types of inheritance.

```

In [2]: class Parent:
        def show(self):
            print("Parent class")

        class Child(Parent):
            def display(self):
                print("Child class")

c = Child()
c.show()
c.display()

```

Parent class

Child class

```

In [3]: class A:
        def methodA(self):
            print("Class A")

        class B:
            def methodB(self):
                print("Class B")

        class C(A, B):
            pass

obj = C()
obj.methodA()
obj.methodB()

```

Class A

Class B

```

In [4]: class Grandparent:
        def show(self):
            print("Grandparent")

        class Parent(Grandparent):
            pass

        class Child(Parent):
            pass

```

```
c = Child()
c.show()
```

Grandparent

```
In [7]: class Parent:
        def show(self):
            print("Parent")

        class Child1(Parent):
            pass

        class Child2(Parent):
            pass

c1 = Child1()
c2 = Child2()

c1.show()
c2.show()
```

Parent

Parent

```
In [6]: class A:
        def showA(self):
            print("A")

        class B(A):
            def showB(self):
                print("B")

        class C(A):
            def showC(self):
                print("C")

        class D(B, C):
            pass

d = D()
d.showA()
d.showB()
d.showC()
```

A

B

C

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the **init** method in Employee to call the parent class's **init** method using the **super()** and then initialize the salary attribute.

```
In [8]: class Person:
        def __init__(self, name, age):
            self.name = name
            self.age = age

        class Employee(Person):
            def __init__(self, name, age, salary):
                super().__init__(name, age)  # Call parent constructor
                self.salary = salary

            def show(self):
                print("Name:", self.name)
                print("Age:", self.age)
                print("Salary:", self.salary)

e = Employee("John", 30, 50000)
e.show()
```

Name: John
Age: 30
Salary: 50000

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

```
In [ ]: class Shape:
        def draw(self):
            raise NotImplementedError("Subclass must implement draw method")

        class Rectangle(Shape):
            def draw(self):
                print("Drawing Rectangle")

        class Circle(Shape):
            def draw(self):
                print("Drawing Circle")

        class Triangle(Shape):
            def draw(self):
                print("Drawing Triangle")

shapes = [Rectangle(), Circle(), Triangle()]

for shape in shapes:
    shape.draw()
```